UNIVERSITY OF TECHNOLOGY, MAURITIUS

SCHOOL OF INNOVATIVE TECHNOLOGIES AND ENGINEERING

DEPARTMENT OF INDUSTRIAL SYSTEM ENGINEERING

INTELLIGENT PERSONALISED FITNESS RECOMMENDATION AND PROGRESS TRACKING SYSTEM

FINAL PROJECT REPORT – DRAFT CHAPTERS 1 TO 5

BY

KURMALLY Mohammad Housein

Student ID: 2403_27576

Module Code: BCNS3006C

This Capstone Project is submitted in partial fulfilment of the requirements for the Bachelor's Degree in Computer Science with Network Security.

SUPERVISOR: Mr Doorgesh Sookarah

July 2026

PROVISIONAL ABSTRACT

This report presents the research, requirements and system design for FitWise, an intelligent personalised fitness recommendation and progress tracking system intended for gym trainers and members. The project addresses the difficulty of translating general fitness information into practical plans that reflect a member's goal, experience, availability, equipment and restrictions. The proposed solution combines a machine-learning classifier for workout-category prediction, a Gemini-assisted plan-generation component, an approved exercise catalogue and deterministic server-side validation. A public website and trainer application will be implemented using Next.js and TypeScript, while a Flutter mobile application will provide member access to assigned plans, workout completion and progress information. Supabase will provide PostgreSQL data storage, authentication, file storage and Row-Level Security, and a lightweight FastAPI service will load and expose the saved Python machine-learning pipeline. The first five chapters establish the project background, critically synthesise relevant literature, define the development methodology and requirements, and specify the complete architecture. No implementation or evaluation results are claimed in this draft; the abstract will be revised after Chapters 6 and 7 are supported by actual model metrics, system tests and user-evaluation evidence.

ACKNOWLEDGEMENTS

The student wishes to express sincere appreciation to the project supervisor, Mr Doorgesh Sookarah, for his guidance and feedback during the definition and planning of this capstone project. Appreciation is also extended to the University of Technology, Mauritius, and the staff of the School of Innovative Technologies and Engineering for providing the academic framework within which the project is being undertaken. The final version of this section will acknowledge any trainers, gym members or test participants who contribute to requirements validation or user acceptance testing.

DRAFT STATUS AND USE

Important
This document is a dissertation-quality draft of Chapters 1 to 5. It intentionally avoids fabricated implementation claims, machine-learning scores, user-test outcomes or deployment results. Sections that depend on completed development must be finalised using evidence collected during implementation and testing.

TABLE OF CONTENTS

Right-click and select Update Field to generate the Table of Contents.

LIST OF FIGURES

Figure 1.1: High-level overview of the proposed FitWise solution

Figure 2.1: Conceptual framework for the FitWise recommendation and feedback loop

Figure 3.1: Iterative development and evaluation methodology

Figure 4.1: Proposed end-to-end FitWise process

Figure 4.2: FitWise use-case model

Figure 5.1: FitWise system architecture

Figure 5.2: Sequence for prediction and plan generation

Figure 5.3: Logical relational data model

Figure 5.4: Machine-learning training and inference pipeline

Figure 5.5: Security boundaries and access-control design

Figure 5.6: Trainer web-application POC screen gallery

Figure 5.7: Gym-member mobile-application POC screen gallery

Figure 5.8: FitWise deployment architecture

LIST OF TABLES

Table 1.1: Project scope and system boundary

Table 2.1: Comparison of representative fitness systems and research prototypes

Table 2.2: Research and development gap analysis

Table 3.1: Project methodology outputs

Table 3.2: Project risk register

Table 4.1: Stakeholder and access analysis

Table 4.2: Functional requirements

Table 4.3: Non-functional requirements

Table 4.4: Prioritised user stories

Table 4.5: Requirements traceability matrix

Table 5.1: Technology stack and design rationale

Table 5.2: Core database entities

Table 5.3: Proposed API endpoints

Table 5.4: Deterministic plan-validation rules

Table 5.5: Access-control matrix

LIST OF ACRONYMS

Acronym	Meaning
API	Application Programming Interface
BMI	Body Mass Index
BPM	Beats Per Minute
CRUD	Create, Read, Update and Delete
F1-score	Harmonic mean of precision and recall
FITT	Frequency, Intensity, Time and Type
GDPR	General Data Protection Regulation
HIIT	High-Intensity Interval Training
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
MVVM	Model–View–ViewModel

Acronym	Meaning
NFR	Non-Functional Requirement
POC	Proof of Concept
RBAC	Role-Based Access Control
RLS	Row-Level Security
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience
WBS	Work Breakdown Structure

CHAPTER 1: INTRODUCTION

1.1 Background and Context

Regular physical activity is associated with substantial physical and mental health benefits, including improved cardiovascular and musculoskeletal health and reduced risk of several non-communicable diseases. Nevertheless, the World Health Organization reported that approximately 31% of adults did not achieve recommended levels of physical activity in 2022, illustrating that access to general information does not automatically translate into sustained behaviour (World Health Organization, 2024). The WHO guidelines emphasise frequency, intensity and duration while recognising that recommendations must be adapted to the individual, particularly where chronic conditions or physical limitations are present (World Health Organization, 2020).

Digital fitness technologies attempt to bridge the gap between broad recommendations and day-to-day action. Mobile applications can deliver reminders, self-monitoring, feedback, instructional media and progress visualisation at relatively low cost. Systematic reviews suggest that app- and tracker-based interventions can produce small-to-moderate improvements in physical activity, with personalisation and continuous feedback associated with stronger outcomes in some studies (Laranjo et al., 2021; Tong et al., 2021). However, the evidence remains heterogeneous, and many applications focus on recording completed activity rather than assisting a user and trainer to create, approve and adapt a coherent workout programme.

Personalisation is important because gym members differ in age, body measurements, training experience, objectives, available time, available equipment, previous performance and reported restrictions. A beginner seeking general fitness should not automatically receive the same exercise volume or intensity as an experienced member pursuing strength. Similarly, a plan that assumes access to a fully equipped gym is unsuitable for a member training at home. When trainers manage many members, collecting these factors, reviewing progress and repeatedly adjusting

weekly plans can become administratively demanding. The challenge is therefore not simply to display exercises, but to transform structured member information into an explainable recommendation that remains reviewable by a trainer.

FitWise is proposed as a multi-client software system that supports this process. It will provide a public website and trainer workspace through Next.js and TypeScript, a member mobile application through Flutter, a PostgreSQL database and authentication services through Supabase, and a Python machine-learning service through FastAPI. A trained classification pipeline will predict a suitable workout category such as Cardio, Strength, HIIT or Yoga. Gemini will then assist in composing a structured weekly plan from an approved exercise catalogue, while the Next.js backend will validate exercise identifiers, session duration, equipment availability, difficulty, reported restrictions and other deterministic constraints. The plan will remain subject to trainer review before assignment.

1.2 Problem Statement

Gym members are exposed to a large quantity of workout advice through websites, social media and general-purpose applications. Although this material may be useful, it is frequently presented as a fixed routine and may not correspond to the member's goal, experience, schedule, equipment or physical limitations. As a result, members may select inappropriate routines, change programmes without a clear basis, or lose motivation when progress is difficult to interpret. The problem is particularly important for beginners, who may not possess sufficient knowledge to distinguish between a popular routine and a suitable one.

Trainers face a related management problem. Member registration, assessment, plan creation, workout monitoring and progress review are often carried out through separate forms, messaging tools or spreadsheets. This fragmentation makes it difficult to maintain a consistent record of the data used for a recommendation, the model or rule version that produced it, the plan approved by the trainer and the member's subsequent completion data. A trainer managing multiple members may therefore spend significant time on repetitive administrative work while still lacking an integrated view of adherence and progress.

Existing commercial systems demonstrate that algorithmic personalisation is feasible, but their internal reasoning, data handling and trainer workflows are generally proprietary. Academic research also shows promise in personalised mobile interventions and LLM-assisted exercise planning, yet concerns remain regarding reliability, safety, privacy, hallucinated content and insufficient expert oversight (Wang et al., 2024; Choi et al., 2026). There is therefore a need for an academically transparent system that combines data-driven prediction, controlled generative assistance, deterministic validation, trainer review, progress tracking and role-based access within one coherent architecture.

1.3 Project Aim

The aim of this project is to design, develop and evaluate an intelligent personalised fitness recommendation and progress tracking system that combines machine-learning-based workout-category prediction with Gemini-assisted workout-plan generation, deterministic validation and trainer review through a Next.js web application and a Flutter gym-member mobile application.

1.4 Project Objectives

1. Analyse and prepare the Gym Members Exercise Dataset for reproducible machine-learning experimentation, including a data-quality and leakage audit.
2. Develop and evaluate classification models for predicting a suitable workout category, such as Cardio, Strength, HIIT or Yoga.
3. Develop a controlled hybrid plan-generation process in which Gemini constructs a structured draft from approved exercises and the Next.js backend enforces deterministic constraints.
4. Develop a transparent progress-estimation method that presents one-, two- and three-month projections with explicit assumptions and limitations.
5. Develop a secure public website and trainer web application using Next.js and TypeScript for member management, assessment, prediction, plan review and progress monitoring.

6. Develop a Flutter mobile application through which gym members can view recommendations, follow assigned plans, record workout completion and review progress.
7. Implement Supabase PostgreSQL, Authentication, Storage and Row-Level Security for persistent data and role/ownership-based access.
8. Integrate the Next.js application, Supabase services, FastAPI inference service, Gemini API and Flutter application into one end-to-end system.
9. Evaluate the machine-learning model, generated-plan validation, software functionality, usability, security, performance and integration, and document the deployed system.

1.5 Research and Development Questions

10. Which classification approach provides the most reliable and practically defensible workout-category prediction on the selected dataset?
11. How can Gemini-assisted generation, an approved exercise catalogue and deterministic validation be combined to produce structured, explainable and reviewable workout plans?
12. To what extent does the completed Next.js, Flutter, Supabase and FastAPI system satisfy its functional, usability, security and integration requirements?

1.6 Proposed Solution Overview

The proposed workflow begins when a trainer creates or activates a member account and records an initial fitness assessment. The Next.js backend validates the assessment, stores the authorised data in Supabase and prepares the features required by the Python model. The FastAPI service loads a saved scikit-learn pipeline once at application start-up and returns the predicted workout category, confidence values and model version. Next.js retrieves approved candidate exercises from Supabase and submits only the necessary profile, prediction and exercise metadata to Gemini using a structured-output schema. The generated draft is treated as untrusted input: server-side validation checks identifiers, durations, repetitions, equipment, difficulty, duplication and exclusion rules before the trainer can approve and assign the plan. The Flutter application then retrieves the approved plan and records workout

completion and progress data, which become available to the member and trainer dashboards.

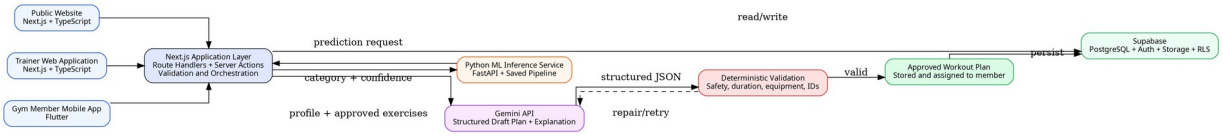


Figure 1.1: High-level overview of the proposed FitWise solution

Source: Author's design

1.7 Scope and System Boundary

Table 1.1: Project scope and system boundary

In Scope	Out of Scope / Constraint
Public FitWise website, trainer web workspace and gym-member mobile application	Medical diagnosis, rehabilitation prescription or emergency monitoring
Supabase email/password authentication with trainer/member access boundaries	Replacement of a qualified doctor, physiotherapist or trainer
ML classification of workout category using a documented dataset and pipeline	Guaranteed body-composition or health outcomes
Gemini-assisted draft plan creation from approved exercises	Unrestricted generation of arbitrary exercises without validation
Deterministic checks for exercise IDs, equipment, duration, difficulty and reported restrictions	Clinical validation for complex health conditions
Trainer review, approval and override with an audit reason	Fully autonomous high-risk decision-making
Workout completion, adherence and progress visualisation	Automatic wearable-device integration unless time permits
POC deployment, technical documentation and controlled user evaluation	Commercial-scale operation or regulatory certification

1.8 Expected Contributions

The principal computing contribution is a transparent hybrid architecture rather than a standalone fitness interface. The project integrates supervised machine learning, a generative model, deterministic business and safety controls, relational data design, role-based security and two user-facing clients. The use of an approved exercise catalogue limits inconsistent naming and permits every generated plan item to be

linked to instructions, equipment and tracking data. The server-side validation layer addresses the fact that structured LLM output may be syntactically correct without being semantically suitable (Google AI for Developers, 2026).

A second contribution is the end-to-end trainer–member workflow. The trainer is not removed from the process; instead, FitWise uses automation to support registration, prediction, plan drafting and monitoring while preserving trainer approval. The member receives a clear weekly plan and records actual completion rather than viewing a one-off recommendation. The resulting data model provides traceability across assessments, predictions, model versions, generated plans, approvals, sessions and progress records.

Finally, the dissertation is intended to contribute an honest evaluation of the selected public dataset and the limitations of progress projection. Post-workout variables will be examined for leakage and unrealistic inference-time availability. Progress estimates will be presented as scenario-based guidance rather than scientifically validated forecasts because the selected dataset does not contain longitudinal observations of the same member over several months.

1.9 Dissertation Structure

Chapter 2 critically reviews physical activity guidance, mobile fitness technologies, recommender systems, machine learning, LLM-assisted exercise planning, progress feedback and privacy. Chapter 3 presents the combined software-engineering and machine-learning methodology, project controls and risk management. Chapter 4 defines the stakeholders, processes, requirements, user stories and traceability. Chapter 5 specifies the architecture, data model, APIs, model pipeline, plan-generation controls, security design, user-interface flows and deployment topology. Chapter 6 will document actual implementation, Chapter 7 will present executed testing and evaluation, Chapter 8 will describe deployment and operation, and Chapter 9 will critically evaluate achievement of the aim and objectives.

1.10 Chapter Summary

This chapter established the need for an integrated, transparent and trainer-supervised fitness recommendation system. It defined the project aim, objectives, questions, scope and expected contributions while distinguishing general fitness guidance from medical decision-making. The next chapter examines the evidence and technical approaches that justify the selected hybrid design.

CHAPTER 2: LITERATURE REVIEW

2.1 Literature Review Strategy

A structured narrative review was undertaken to connect the fitness domain with the computing decisions required by FitWise. Sources were identified through PubMed, IEEE Xplore, ACM publications, Google Scholar, SpringerLink, ScienceDirect and official standards or product documentation. Searches combined terms including personalised fitness, physical activity application, mHealth intervention, exercise recommendation, hybrid recommender, machine-learning classification, large language model exercise planning, health-data privacy, API security, Next.js, Supabase, Flutter and FastAPI. Priority was given to peer-reviewed systematic reviews, meta-analyses, recognised exercise guidance and primary technical documentation. Recent work from 2021–2026 was emphasised, while foundational recommender-system and behaviour-change studies were retained where they define concepts still used in current research.

The review was not designed as a clinical systematic review and does not claim exhaustive coverage. Sources were selected for their relevance to the proposed system and assessed according to publication type, methodological transparency, sample size or evidence base, direct relevance, and the authority of the issuing organisation. Commercial application documentation was used only to compare publicly described features, not to make unverified claims about proprietary algorithms.

2.2 Physical Activity, Exercise Prescription and Fitness Monitoring

The WHO guidelines describe physical activity recommendations in terms of frequency, intensity and duration and advise adults to undertake regular aerobic and muscle-strengthening activity (World Health Organization, 2020). Exercise planning is therefore not limited to selecting a label such as Cardio or Strength. A useful system must eventually translate a broad category into sessions that balance type, duration, intensity and recovery. Garber et al. (2011) similarly describe exercise programmes as combinations of cardiorespiratory, resistance, flexibility and

neuromotor activity, reinforcing the need to consider different fitness components rather than maximising one metric.

Fitness monitoring commonly uses body measurements, session completion, heart rate, duration and estimated energy expenditure. Ainsworth et al. (2011) demonstrated that activities have different energy costs, but values such as calories burned are estimates influenced by body mass, duration, intensity and measurement method. Consequently, FitWise should not treat any single value as a definitive indicator of health or progress. Measurements must also be interpreted according to when they are available. Resting heart rate and stated workout frequency may be collected during onboarding, whereas average exercise heart rate, session duration and calories burned are normally known after a workout. Mixing these variables in a new-member prediction creates a methodological risk that must be addressed in the ML design.

The project therefore uses exercise guidance as a framework and keeps a human reviewer in the loop. Reported restrictions can influence exclusion or substitution, but the application will not diagnose a condition. A trainer must review plans where a member declares pain, injury or another limitation, and the interface must provide an explicit statement that the recommendation is general fitness guidance.

2.3 Digital Health and Mobile Fitness Applications

Mobile fitness applications can deliver self-monitoring, feedback and reminders in the context in which behaviour occurs. Emberson et al. (2021) found that smartphone interventions produced positive outcomes in a proportion of reviewed studies across physical, weight-related and psychological measures. Laranjo et al. (2021) reported a small-to-moderate positive effect from apps and activity trackers and identified personalisation and text messaging as potentially important features. These results support the inclusion of progress feedback and notifications, but they also indicate that technology alone does not guarantee long-term adherence.

Personalisation appears to improve relevance. Tong et al. (2021) synthesised personalised mobile interventions and found preliminary evidence of a moderate positive effect on lifestyle behaviours. Interventions using system-captured data were associated with stronger effects than those relying only on self-report. FitWise

initially depends on trainer-entered and member-entered information, so the limitation of self-reported data should be acknowledged. The design partially mitigates this through structured validation, recorded workout completion and the possibility of comparing initial assessments with later measurements.

Application quality is uneven. Paganini et al. (2021) found substantial variation in physical-activity apps and assessed both quality and privacy/security measures. Davergne et al. (2023) found that personalised exercise-video applications may improve physical function and exercise confidence, but the quality of evidence was low. These findings suggest that a convincing final-year project should evaluate task completion, clarity and trust rather than using attractive mockups as proof of usability. FitWise will therefore require task-based user acceptance testing and not only interface screenshots.

Behaviour-change theory also supports goal setting, self-monitoring and feedback. The Behaviour Change Wheel describes behaviour through capability, opportunity and motivation (Michie, van Stralen and West, 2011), while the Behaviour Change Technique Taxonomy identifies goal setting, feedback and self-monitoring among commonly used techniques (Michie et al., 2013). FitWise incorporates these principles through visible goals, weekly completion, progress trends, trainer feedback and reassessment triggers. Gamification features such as badges may support engagement, but they are secondary because the core research contribution lies in recommendation, integration and evaluation.

2.4 Recommender Systems and Personalisation

Recommender systems reduce information overload by selecting or ranking items according to user data, item characteristics and interaction history. Collaborative filtering learns from similarities between users or behaviours, while content-based methods match item properties to a user profile. Knowledge-based and rule-based methods apply explicit domain relationships or constraints, and hybrid systems combine approaches to compensate for individual weaknesses (Adomavicius and Tuzhilin, 2005; Burke, 2002; Ricci, Rokach and Shapira, 2011).

Collaborative filtering is not appropriate as the primary method for the present dataset because it does not contain user ratings, repeated preference histories or interactions between many identifiable members and many workout plans. A content- or knowledge-based approach is more defensible: member characteristics and a predicted workout category can be matched to exercises described by category, difficulty, equipment and constraints. The classification model provides a data-driven signal, while the catalogue and validation logic preserve explicit control.

The selected design is therefore hybrid at three levels. First, supervised ML predicts the broad workout category. Second, Gemini synthesises the member context and candidate exercises into a structured weekly draft. Third, deterministic checks and trainer review decide whether the draft is acceptable. Research on hybrid clinical decision systems shows that machine learning and rule-based reasoning can be combined in multiple architectures to achieve both pattern recognition and explicit control (Kierner, Kucharski and Kierner, 2023). Although FitWise is not a clinical system, the same design principle is useful because exercise-plan constraints should remain inspectable.

Evaluation must consider more than overall accuracy. Herlocker et al. (2004) argued that recommender evaluation should reflect the task and user experience rather than a single generic metric. For FitWise, model performance, plan validity, trainer acceptance, task completion and perceived clarity are separate outcomes. A high classification score does not prove that a generated weekly plan respects equipment or duration, and a valid plan does not prove that users can understand the mobile workflow.

2.5 Machine Learning for Fitness Prediction

The selected Gym Members Exercise Dataset contains 973 records and 15 variables, including demographic characteristics, body measurements, heart-rate metrics, workout frequency, session duration, calories burned, experience level and the target Workout_Type (Khorasani, 2024). Its four target categories make the problem suitable for multi-class classification. Candidate algorithms include logistic regression, decision trees, random forests, support vector machines and gradient

boosting. Logistic regression provides an interpretable baseline; decision trees expose explicit splits but may overfit; random forests can capture non-linear interactions and reduce individual-tree variance; support vector machines can perform well in medium-sized feature spaces but may require scaling and provide less direct interpretability.

The project must avoid data leakage and unrealistic input assumptions. Workout_Type must never appear among the predictors. Duplicate records should be removed before splitting so that equivalent rows do not appear in both training and test sets. More importantly, variables recorded during or after a workout—such as Avg_BPM, Max_BPM, Session_Duration and Calories_Burned—may reveal the workout that was already performed rather than help select a workout before it occurs. A model trained on those fields could achieve a strong score while failing the real application scenario. FitWise will therefore conduct a feature-availability audit and compare a new-member feature set with an optional returning-member feature set if sufficient historical data are available.

The preprocessing and classifier should be saved as one pipeline so that categorical encoding, scaling and feature order remain identical during training and inference. Scikit-learn provides tools for pipelines and evaluation, but persisted joblib or pickle-based artefacts should only be loaded from trusted sources and require compatible Python and dependency versions (Scikit-learn Developers, 2026). FastAPI lifespan events can load the model once before requests are accepted, avoiding model reload for each prediction (FastAPI, 2026a).

Model selection should use a stratified held-out test set, cross-validation and class-level metrics. Accuracy alone can hide poor performance on one category. Macro precision, macro recall and macro F1 give equal weight to each class, while the confusion matrix reveals systematic category errors. Confidence values must be described as model scores rather than probabilities of safety or guaranteed success. The final report should also include a model card recording intended use, training data, features, performance, limitations and the model version stored with every prediction.

2.6 Generative AI and Controlled Workout-Plan Generation

A traditional rule engine could map every combination of goal, level, days, equipment and restriction to a fixed template, but the number of combinations grows quickly and results can become repetitive. Large language models offer a more flexible mechanism for combining natural-language constraints and producing explanations. PlanFitting demonstrated that an LLM-driven conversational system could help users articulate constraints and create actionable weekly exercise plans; the design grounded generation in an external exercise dataset to reduce irrelevant or hallucinated exercises (Shin, Hsieh and Kim, 2025).

However, generative output cannot be treated as authoritative. Wang et al. (2024) found that reliability and bias were dominant concerns in research on conversational LLMs in health care. Recent expert evaluation of Gemini-generated exercise prescriptions found that structured prompting improved some aspects of safety and guideline alignment but still assumed expert involvement (Choi et al., 2026). A 2026 systematic review concluded that LLMs should be treated as assistive tools rather than replacements for professional exercise decision-making (He et al., 2026). These findings strongly support FitWise's human-in-the-loop and validation design.

Gemini structured outputs can enforce a JSON schema, enabling the application to request fixed fields such as plan name, days, exercise identifiers, sets, repetitions, duration and rest. Google's documentation nevertheless states that schema compliance does not guarantee semantically correct values and recommends application-level validation (Google AI for Developers, 2026). FitWise will therefore permit Gemini to choose only from candidate exercise IDs supplied by the backend. Unknown or inactive IDs, duplicate exercises, incompatible equipment, out-of-range values and restricted movements will be rejected. Where a draft fails validation, the backend can issue one constrained repair request or return the errors for trainer review.

This approach is more defensible than replacing the catalogue entirely with generated text. A controlled catalogue gives each exercise a stable identifier and links it to instructions, difficulty, equipment, muscle group and media. It also makes workout history and analytics reliable because the same exercise is not recorded under several

names. Generative AI is used for composition and explanation, while the application remains the source of truth.

2.7 Progress Estimation and Feedback

Progress tracking supports reflection and may improve motivation, but progress estimation must be framed carefully. The selected dataset is cross-sectional and does not track the same people at one-, two- and three-month intervals. It cannot therefore train a scientifically validated longitudinal forecasting model. A projection based on current weight, stated goal, workout frequency and expected adherence is an assumption-based estimate rather than evidence that a specific result will occur.

FitWise will separate actual progress records from projected values. Actual records will include dated measurements, completed sessions, adherence and optional calories or body-fat information. The projection will apply a transparent, configurable change rate and display the assumptions used, including the adherence scenario and the absence of dietary modelling. When a new measurement is recorded, the dashboard can compare actual and projected trends and recalculate future guidance. The system will avoid language such as “guaranteed result” or “accurate prediction”.

Feedback must also remain actionable. The trainer dashboard should highlight missed sessions, low adherence and milestone achievement, while the member dashboard should answer what to do today, how the current week is progressing and how recent values compare with the starting point. This aligns the progress module with self-monitoring and feedback techniques identified in behaviour-change research (Michie et al., 2013).

2.8 Security, Privacy and Ethical Considerations

FitWise processes data that may include body measurements, heart-rate values, fitness goals and restrictions. Such data can be sensitive even when the project is not providing health care. The European Commission identifies data concerning health as a special category under the GDPR, and the European Data Protection Board emphasises purpose limitation, data minimisation, storage limitation, integrity and confidentiality (European Commission, n.d.; European Data Protection Board, n.d.).

The project should therefore collect only fields required for recommendation and progress features, explain their purpose and avoid using real personal data in demonstrations without consent.

Security concerns can reduce adoption of mHealth applications. Aljedaani and Babar (2021) identified developer knowledge and organisational practices as important challenges in secure mHealth development, while a later systematic review found that users repeatedly express concern about confidentiality, privacy and security (Alhuwail et al., 2024). FitWise will use Supabase Authentication, server-side token verification and PostgreSQL Row-Level Security. Members should access only their own records; trainers should access only assigned members; service-role credentials and the Gemini key must remain on trusted servers.

API design must also address broken object-level authorisation, broken authentication, unrestricted resource consumption and unsafe consumption of third-party APIs, all of which appear in the OWASP API Security Top 10 (OWASP, 2023). Every endpoint that accepts a member or plan identifier must verify the caller's role and relationship to that record. Requests to Gemini and FastAPI must be server-to-server, time-limited and validated. Rate limiting and bounded schema sizes are required for generation endpoints because they can consume external quota and compute resources.

Ethically, the system should be positioned as decision support. Explanations should disclose that recommendations are generated from a limited public dataset and approved catalogue. Trainers must be able to reject or edit a draft, and the audit record should capture who approved a plan. Users should have a way to correct their data, and test participants should receive a clear explanation of the study, data use and voluntary nature of participation.

2.9 Existing Systems and Comparative Analysis

Representative systems illustrate the current fitness-technology landscape. Fitbod publicly describes workouts tailored to goals, experience, equipment, duration, workout history and recovery, with exercise demonstrations and adaptive progression (Fitbod, 2026). Freeletics describes an AI Coach that adjusts plans according to

onboarding information, performance and post-session feedback (Freeletics, 2026). Strava emphasises activity recording, goals, training plans and longitudinal performance tracking, particularly for endurance activities (Strava, 2026). These systems demonstrate strong consumer functionality, but their internal models and validation rules are proprietary and they are not designed as transparent academic trainer–member platforms.

Academic prototypes provide complementary insights. P3FitRec explores privacy-preserving recommendations from wearable fitness data (Liu et al., 2022), while PlanFitting uses an LLM and external exercise knowledge to support iterative plan creation (Shin, Hsieh and Kim, 2025). FitWise does not attempt to reproduce the scale or sensor integration of these systems. Its contribution is the transparent integration of a public classification experiment, approved catalogue, structured LLM generation, deterministic validation, trainer approval and separate trainer/member clients.

Table 2.1: Comparison of representative fitness systems and research prototypes

System / Study	Personalisation	Trainer Support	Progress / Feedback	Transparency / Limitation
Fitbod	Algorithmic workouts based on goals, history, recovery, time and equipment	Primarily self-service	Workout reports, recovery and performance metrics	Strong adaptive features; proprietary logic and consumer focus
Freeletics	AI Coach adapts journeys from onboarding, performance and feedback	Digital coach rather than gym-trainer workspace	Feedback-driven plan adaptation	Proprietary system; subscription-based consumer product
Strava	Goals, plans and analytics informed by recorded activities	Coaches/clubs possible but not central to plan generation	Extensive activity history and performance trends	Strong tracking; less focused on multi-type gym plan generation
P3FitRec (Liu et al., 2022)	Deep-learning recommendation from wearable data with privacy focus	No integrated gym-trainer approval workflow described	Predicts distance, speed and heart-rate sequences	Research prototype requiring large sensor histories
PlanFitting (Shin et al., 2025)	LLM conversation plus external exercise knowledge	Expert evaluation, but not a full trainer management platform	Iterative plan refinement	Promising planning support; generation requires grounding and oversight
Proposed FitWise	ML category prediction + Gemini plan draft + deterministic validation	Dedicated trainer web app with approval and override	Member completion, adherence and progress dashboards	Academic POC; transparent architecture, limited dataset and no medical claims

2.10 Research and Development Gap

The literature indicates that mobile interventions can support activity and that personalisation can improve relevance, but four gaps remain directly relevant to this project. First, many applications either provide broad content libraries or use proprietary recommendation logic that cannot be examined in an academic evaluation. Second, exercise planning research often focuses on a single user-facing tool and does not integrate trainer approval, member completion and traceable model evidence. Third, generative systems can produce flexible plans but require grounding, validation and expert oversight. Fourth, progress projections are often presented persuasively despite limited longitudinal evidence.

Table 2.2: Research and development gap analysis

Observed Gap	Evidence	FitWise Response	Evaluation
Generic or opaque workout recommendations	Mobile and commercial systems frequently hide the basis of selection	Documented ML features, model version, confidence and plan rationale	Model metrics, explanation review and trainer acceptance
Weak integration between trainer management and member execution	Most studies focus on one client or one intervention component	Next.js trainer application and Flutter member application share the same Supabase data	End-to-end integration tests and UAT tasks
LLM flexibility without dependable semantic safety	LLM literature reports reliability, hallucination and oversight concerns	Approved exercise IDs, JSON schema, deterministic validator and trainer approval	Normal, boundary, conflict and restricted-profile tests
Unclear treatment of sensitive fitness data	mHealth studies report privacy and security concerns	Supabase Auth, RLS, least privilege, data minimisation and audit logs	Negative authorisation tests and privacy review
Progress forecasts unsupported by longitudinal data	Selected dataset contains sessions, not repeated monthly outcomes	Transparent scenario projection separated from actual progress	Sensitivity checks, disclaimer and actual-versus-estimated comparison

2.11 Conceptual Framework

The conceptual framework links personalisation inputs to a controlled feedback loop. A member profile and contextual constraints enter the classification stage. The category prediction and approved exercise catalogue inform Gemini-assisted drafting. Deterministic validation and trainer review transform the draft into an assigned plan.

Completion and progress records then inform dashboards and later reassessment. This separation makes it possible to test each component independently while preserving an end-to-end user journey.



Figure 2.1: Conceptual framework for the FitWise recommendation and feedback loop

Source: Author's design

2.12 Chapter Summary

The literature supports personalised mobile feedback and hybrid recommendation, but it also exposes limitations in evidence quality, data availability, generative reliability and privacy. These findings justify a trainer-supervised architecture in which ML and Gemini provide assistance while approved data, deterministic checks and explicit access control remain authoritative. Chapter 3 translates these conclusions into a reproducible development and evaluation methodology.

CHAPTER 3: METHODOLOGY AND PROJECT MANAGEMENT

3.1 Methodology Overview

The project will use a combined methodology because it contains both software-development and machine-learning research activities. The software will be developed iteratively through short vertical slices, while the ML component will follow a reproducible experimental sequence of data understanding, preparation, modelling, evaluation and deployment. These two workstreams will converge early: a simple prediction endpoint and a minimal member workflow will be integrated before all screens are completed. This reduces the risk of producing attractive but disconnected interfaces.

The methodology has eight stages: requirements and literature review; POC and UX prototyping; data preparation and ML experiments; architecture and database design; vertical-slice implementation; integration and security hardening; testing and user evaluation; and deployment and documentation. Feedback can move the project back to earlier stages. For example, a failed RLS test may require a database-policy redesign, and an unrealistic model feature may require repeating preprocessing and experiments.

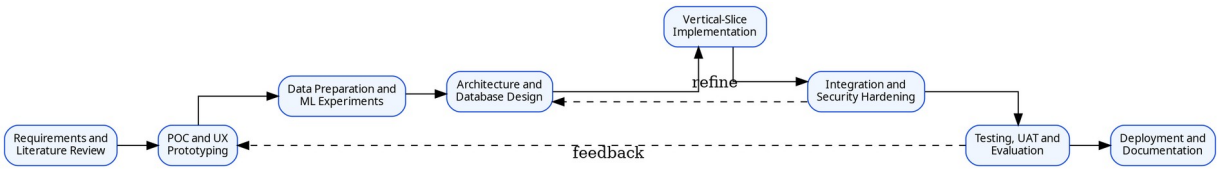


Figure 3.1: Iterative development and evaluation methodology

Source: Author's design

3.2 Requirements-Gathering Approach

The initial requirements were derived from the approved ISWP, the BCNS project handbook, literature on fitness applications and recommendation systems, supervisor discussions and iterative mockup review. The POC screens were used as elicitation artefacts to expose missing actions, duplicate registration flows and excessive scope.

Requirements will be refined through walkthroughs with the supervisor and, where available, one or more trainers or representative gym members. The final report will distinguish clearly between methods actually performed and methods that were merely considered.

No claim is made at this stage that formal interviews or questionnaires have been completed. If user input is collected, the student will record participant role, consent, questions, anonymised findings and the requirements that changed. Otherwise, the requirements source will remain document analysis, literature, supervisor feedback and prototype inspection.

3.3 Software Development Lifecycle

An iterative Agile-style lifecycle will be used without claiming full organisational Scrum. Work will be maintained in a prioritised backlog and implemented in time-boxed increments. Each increment should produce a reviewable artefact such as a database migration, authenticated page, API endpoint, model experiment or Flutter feature. Git branches and pull requests or clearly documented commits will provide version history. Core requirements will be completed before optional features such as badges, messaging or advanced social login.

The preferred implementation order is a thin end-to-end slice: trainer authentication; member creation; assessment persistence; FastAPI prediction; result storage; approved exercise retrieval; Gemini draft; deterministic validation; trainer approval; Flutter plan display; completion write-back; and dashboard update. Once this chain works, additional screens and analytics can be added with lower integration risk.

3.4 Dataset Selection and Data Preparation Method

The Gym Members Exercise Dataset was selected because it contains 973 observations and four workout categories with demographic, body and workout variables under an Apache 2.0 licence (Khorasani, 2024). The data-preparation process will record the original file checksum or version, variable names and types, missing values, duplicates, implausible ranges, target distribution and correlations.

Categorical predictors will be encoded within a scikit-learn ColumnTransformer, and numeric scaling will be applied where required by the candidate model.

A dedicated leakage and feature-availability audit will be completed before modelling. The audit will classify each variable as available before the first recommendation, available only after workout history exists, derived from another variable or unsuitable. The initial new-member model should prioritise age, gender, height, weight, BMI, body-fat percentage, resting BPM, planned workout frequency and experience level. Post-workout metrics will be excluded from the primary onboarding model unless the final use case explicitly defines a returning-member prediction and maintains a separate evaluation.

After duplicate removal, the dataset will be divided using a stratified train/test split with a fixed random seed. All preprocessing will be fitted only on training folds. The held-out test set will remain untouched until model selection is complete. Class distributions will be reported, and synthetic oversampling will not be introduced unless class imbalance is demonstrated and the method is evaluated without contaminating the test set.

3.5 Machine-Learning Experimental Method

The experimental comparison will include a majority-class or simple baseline and at least logistic regression, decision tree, random forest and support vector machine classifiers. Gradient boosting may be included if time permits. Five-fold stratified cross-validation will estimate validation stability, and hyperparameters will be tuned through a bounded grid or random search. Candidate models will be compared using accuracy, macro precision, macro recall, macro F1, training time, inference time and interpretability.

The final model will not automatically be the one with the highest validation accuracy. Selection will consider test-set generalisation, per-class errors, confidence availability, complexity and deployment requirements. The complete preprocessing and classifier pipeline will be exported using a trusted persistence method such as joblib and accompanied by dependency versions, feature schema, model version and

training metadata. The FastAPI service will reject missing, extra or out-of-range fields before inference.

3.6 Hybrid Workout-Plan Generation and Validation Method

The plan-generation module will use a controlled hybrid procedure rather than allowing Gemini to generate an unrestricted plan. First, Next.js will query Supabase for active exercises compatible with the predicted category, member level and available equipment. Each candidate will include a stable identifier and only the metadata required for generation. Second, a server-side prompt will state the member context, maximum days, session duration, target category and restrictions and require an output that matches a Zod/JSON schema. Third, the response will be parsed and subjected to deterministic validation.

Validation rules will include unknown exercise rejection, inactive exercise rejection, equipment compatibility, difficulty compatibility, duration limits, bounds for sets/repetitions/rest, duplicate control, weekly session count and exclusion mappings for reported restrictions. Safety and ownership checks will have higher priority than goal preference. A trainer will review the valid draft and may edit, reject or approve it. Overrides will be stored with a reason to preserve auditability. The final report will test at least beginner, intermediate, advanced, restricted, boundary and conflicting profiles.

3.7 Progress-Estimation Method

Because the dataset lacks longitudinal targets, the progress module will use transparent scenario projection rather than a fitted forecasting model. A configurable expected monthly change will be combined with the current value and an adherence factor. For a decreasing weight goal, a simple draft formulation is: projected change = base monthly change × adherence factor × elapsed months. The exact rate will be conservative, documented and reviewed with the supervisor; it will not be represented as a clinical prescription.

The dashboard will display the baseline, target, one-, two- and three-month projection, actual dated records and the assumptions used. Sensitivity scenarios can

demonstrate the effect of lower, expected and higher adherence. When actual values are added, the system will calculate deviation from the projection and update the next estimate. All outputs will include a statement that results vary with nutrition, sleep, health, measurement error and individual response.

3.8 Testing and Evaluation Method

Evaluation will cover the ML model, Next.js and FastAPI APIs, database security, Flutter and web workflows, Gemini output validation, usability and performance. Unit tests will verify data transformations, plan validators and utility functions. API tests will cover valid and invalid payloads, authentication, role checks, duplicate actions and persistence. Integration tests will trace the complete prediction-to-plan and completion-to-progress flows. System tests will execute realistic trainer and member scenarios.

User acceptance testing will use task scripts rather than a general satisfaction question. Representative tasks will include registering a member, generating and interpreting a prediction, approving a plan, locating today's workout, recording completion and interpreting progress. Measures will include completion, errors, time and a short usability rating. ML evaluation will use the held-out metrics and confusion matrix described in Section 3.5. Security testing will attempt expired tokens, member access to trainer endpoints, cross-member access, malformed identifiers and direct RLS-protected queries.

3.9 Ethics, Privacy and Safety

Only data required by the implemented features will be collected. Demonstration accounts will use synthetic identities. Any user-testing participants will receive an information statement and provide consent, and results will be reported using anonymous participant codes. The project will not collect medical records or claim to treat injury. A member who reports a serious condition will be directed to seek qualified professional advice rather than receive an autonomous plan.

Authentication and authorisation will be separate concerns: Supabase Auth will establish identity, while application role checks and RLS will restrict access. The

Gemini prompt will avoid direct identifiers and send only the minimum profile context required for plan construction. API keys and service-role credentials will be stored in server environment variables and excluded from source control and client bundles.

3.10 Quality Assurance

Quality assurance will include TypeScript strict mode, ESLint, formatted code, modular services, Dart static analysis, Python linting where practical, automated tests and reproducible dependency files. Database changes will be captured as migrations. API schemas will be shared or documented so that Flutter and Next.js use consistent field names. Errors returned to users will be controlled and will not reveal stack traces or secrets.

An evidence folder will retain dataset reports, model experiment outputs, API screenshots, database policies, test logs, defects, deployment records and supervisor decisions. This supports the handbook expectation that the final report and demonstration faithfully represent completed work. Every claim of security, performance or usability will be tied to a test rather than asserted without evidence.

3.11 Project Plan and Progress Control

The original ISWP planned Firebase and a web frontend separated from the backend. The architecture has since changed to Next.js and TypeScript for the public website, trainer interface and main backend; Supabase for PostgreSQL, authentication, storage and RLS; Flutter for mobile; FastAPI for Python inference; and Gemini for controlled plan drafting. This change reduces duplicated web/backend code and provides a relational data model that is suitable for plans, exercises, sessions and progress records. The change must be recorded in the project diary and updated Gantt chart rather than silently replacing the original plan.

Progress will be reviewed against milestones rather than screen count. The critical milestones are database and authentication baseline, model experiment completion, prediction integration, generated-plan validation, Flutter plan completion, security

testing and deployment. Buffer time must remain for report revision, model reruns and demonstration preparation.

Table 3.1: Project methodology outputs

Phase	Key Output	Completion Evidence
Research and requirements	Critical literature synthesis, scope and traceable requirements	Chapter drafts, requirements catalogue and supervisor feedback
Data and ML	Leakage audit, reproducible experiments and saved pipeline	Notebook/script, metrics, confusion matrix and model card
Data and security design	Supabase schema, RLS policies and access matrix	Migrations and positive/negative policy tests
Vertical slice	Member assessment to prediction to approved plan	API logs, database records and UI evidence
Mobile integration	Assigned plan, completion and progress update	Device/emulator test and shared data evidence
Evaluation and deployment	System, UAT, security and deployment results	Test report, defect log, URLs/APK and manuals

3.12 Risk Register

Table 3.2: Project risk register

Risk	Likelihood	Impact	Mitigation / Contingency
Data leakage produces unrealistic model performance	Medium	High	Audit feature timing; remove post-workout variables from onboarding model; test duplicates before split.
Public dataset is synthetic or insufficiently representative	High	High	State limitation, avoid medical claims, evaluate model only within dataset scope and plan future real-world validation.
Gemini returns valid JSON with unsuitable values	High	High	Approved exercise IDs, strict schema, deterministic validation, one bounded repair attempt and trainer approval.
Next.js, Flutter and FastAPI integration is delayed	High	High	Implement one thin vertical slice early and freeze optional screens until it works.
Supabase RLS is misconfigured	Medium	High	Enable RLS on every exposed table; test anon, member, other-member and

Risk	Likelihood	Impact	Mitigation / Contingency
			trainer scenarios.
Scope becomes too large	High	High	Prioritise core web/mobile workflow; defer badges, chat and advanced notifications.
External API or internet failure affects demo	Medium	High	Store an approved sample plan, provide controlled fallback mode and retain a short backup recording.
Insufficient UAT participants	Medium	Medium	Plan recruitment early; use task-based qualitative evidence and avoid statistical over-generalisation.
Secrets are committed or exposed in clients	Low	High	Environment variables, .gitignore, secret scanning and key rotation if exposure occurs.

3.13 Chapter Summary

The methodology combines iterative software engineering, reproducible ML experimentation and controlled generative-AI integration. It prioritises an early vertical slice, explicit leakage controls, deterministic plan validation, privacy, traceable testing and evidence collection. Chapter 4 now translates this method into testable stakeholder and system requirements.

CHAPTER 4: REQUIREMENTS ANALYSIS

4.1 Stakeholders and User Roles

The core operational stakeholders are the trainer and gym member. The trainer creates or activates a member account, confirms assessment information, requests and interprets predictions, reviews generated plans and monitors progress. The member uses the Flutter application to view the assigned recommendation and weekly plan, record completion and inspect personal progress. A system administrator role should be implemented only if operational configuration genuinely requires it; otherwise, administrative functions remain deployment responsibilities rather than a third business role.

Table 4.1: Stakeholder and access analysis

Stakeholder / Role	Primary Needs	Responsibilities	Access Boundary
Trainer	Manage assigned members, assessments, predictions, plans and progress	Verify input, review AI output, approve/override plans and respond to alerts	Trainer-only functions and records of assigned members
Gym Member	View own recommendation and plan, record workouts and progress	Provide accurate information and report pain or limitations	Own profile, plans, sessions and progress only
System Maintainer	Configure deployment, exercise catalogue and model version	Protect secrets, migrations, logs, backups and releases	Trusted server/admin environment; not a normal member role
Supervisor / Examiner	Evidence of substantial, tested and ethically scoped computing work	Assess report, code and demonstration	Documentation and controlled demonstration access

4.2 Current and Proposed Process

In the current general process, a member may receive advice from internet sources or a trainer, record progress inconsistently and communicate changes through separate channels. The trainer may maintain member details, plans and progress in unrelated documents. There is limited traceability between the data used for a recommendation and later outcomes, and repeated plan creation can be time-consuming.

The proposed process creates a single controlled record. The trainer signs in, creates the member and assessment, and requests a prediction. Approved exercises are

supplied to Gemini, the draft is validated and the trainer approves the plan. The member views and completes the plan in Flutter, and the shared database updates adherence and progress for both roles. Failed prediction, generation or validation steps return a controlled error without assigning a partial plan.

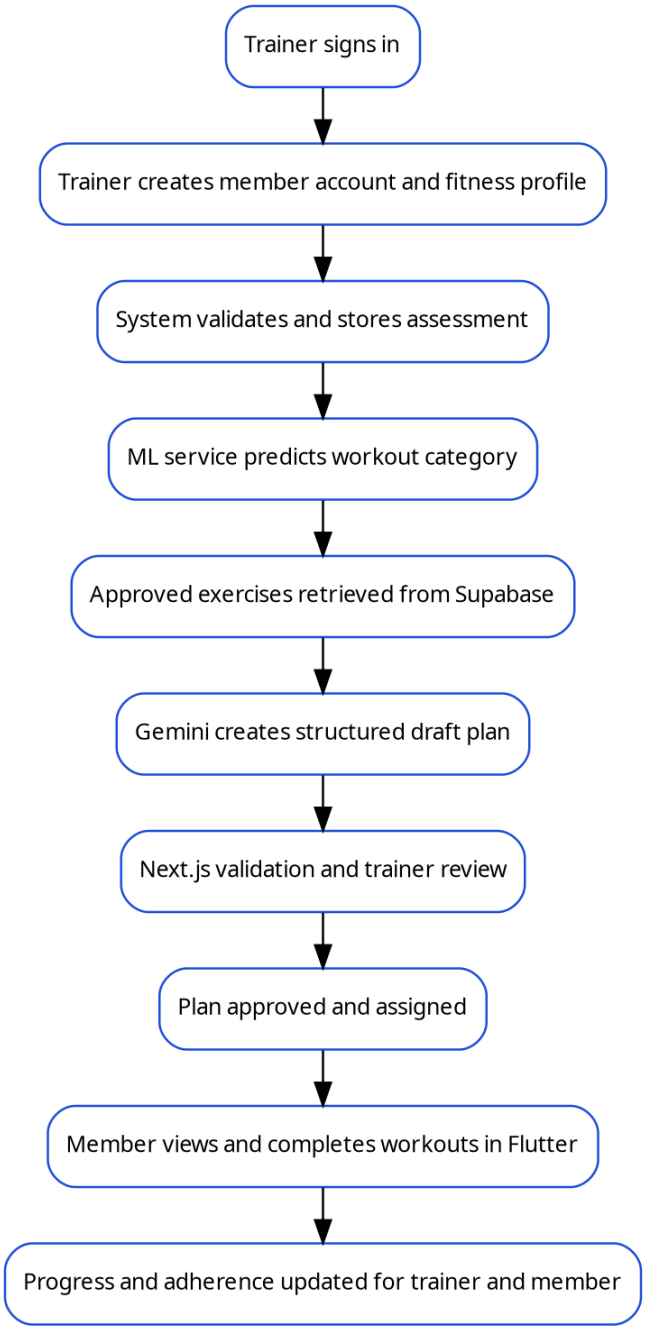


Figure 4.1: Proposed end-to-end FitWise process

Source: Author's design

4.3 Functional Requirements

Table 4.2: Functional requirements

ID	Requirement	Priority	Acceptance Criterion
FR-01	The public website shall present the FitWise purpose, features, disclaimer and access links.	Must	Pages are accessible and navigation reaches login/download information.
FR-02	The trainer and member shall authenticate using Supabase Auth.	Must	Valid credentials create a session; invalid, expired or disabled access is rejected.
FR-03	The trainer shall create a unique member account/profile.	Must	Required fields are stored and duplicate email/account creation is rejected.
FR-04	The trainer shall record and update a member fitness assessment.	Must	Validated measurements, goals, availability, equipment and restrictions are persisted.
FR-05	The system shall request and store a workout-category prediction.	Must	Prediction, confidence, model version, assessment and timestamp are saved.
FR-06	The system shall retrieve only active approved exercises compatible with the plan context.	Must	Candidate list excludes inactive and clearly incompatible catalogue items.
FR-07	The system shall generate a structured draft plan using Gemini.	Must	Response matches the required schema or returns a controlled generation error.
FR-08	The Next.js backend shall validate every generated plan before it is shown for approval.	Must	Unknown IDs, invalid bounds, unavailable equipment and excluded exercises are rejected.
FR-09	The trainer shall review, edit, approve, reject or regenerate a draft plan.	Must	Only an approved plan can be assigned; override reason is recorded.
FR-10	The member shall view the current recommendation and approved weekly plan.	Must	Member sees only own assigned plan with exercise details.
FR-11	The member shall start and complete a workout session.	Must	Session, exercises, actual repetitions/duration and completion status are stored.
FR-12	The trainer and member shall view authorised progress and adherence information.	Must	Dashboards refresh from stored assessments, sessions and progress records.
FR-13	The trainer shall record new measurements and compare actual with projected progress.	Should	New record updates charts and recalculates the projection.
FR-14	The system shall notify users of important plan, reminder or adherence events.	Should	Relevant notification is stored and displayed to the correct user.
FR-15	The system shall maintain audit records for sensitive actions.	Must	Creation, approval, override and role-sensitive changes record actor and timestamp.
FR-16	The member and trainer shall manage profile and notification preferences.	Should	Permitted fields update without bypassing ownership/role rules.
FR-17	The system maintainer shall	Should	Only trusted administrative

ID	Requirement	Priority	Acceptance Criterion
	manage approved exercises and media.		operation can activate/deactivate catalogue items.
FR-18	The system shall provide controlled loading, empty and error states.	Must	Client displays understandable feedback without exposing secrets or stack traces.

4.4 Non-Functional Requirements

Table 4.3: Non-functional requirements

ID	Category	Requirement	Measure / Target
NFR-01	Security	Users shall access only functions and records permitted by role and ownership.	All negative role/ownership tests pass; no cross-member disclosure.
NFR-02	Privacy	Only necessary fitness data shall be collected and purpose/disclaimer text displayed.	Data inventory completed; unnecessary identifiers excluded from Gemini prompts.
NFR-03	Performance	Core non-generation API requests shall respond promptly under POC load.	Median server response target <2 s locally/hosted; generation separately measured.
NFR-04	Reliability	Failed external calls shall not create partial or approved records.	Transactions/statuses preserve consistent state and retries are bounded.
NFR-05	Usability	Core trainer/member tasks shall be understandable without developer assistance.	Target task completion ≥80% in small POC UAT; issues documented.
NFR-06	Maintainability	The system shall use modular services, typed schemas and migrations.	Repository structure, linting and setup documentation reviewed.
NFR-07	Compatibility	Trainer web app shall support modern desktop browsers; mobile POC shall run on target Android device/emulator.	Defined browser/device matrix passes core scenarios.
NFR-08	Auditability	Predictions and plan approvals shall be traceable.	Model version, prompt/plan version, actor and timestamps recorded.
NFR-09	AI robustness	Gemini output shall never be accepted solely because it is valid JSON.	All plans pass server validator and trainer approval.
NFR-10	Accessibility	Interfaces shall use readable contrast, labels and touch targets.	Manual accessibility checklist and keyboard/mobile review completed.
NFR-11	Scalability	POC design shall avoid hard-coded member-specific logic and support indexed queries.	Relational schema and filtered RLS queries documented.
NFR-12	Recoverability	Development and deployment configuration shall support reconstruction.	Migrations, environment template, model metadata and README available.

4.5 Use-Case Model

The use-case model separates the trainer's supervisory functions from the member's execution and self-monitoring functions. Both roles authenticate and manage permitted profile settings, but only the trainer manages assessments, predictions and approvals. The member cannot call trainer-only generation endpoints or retrieve another member's records.

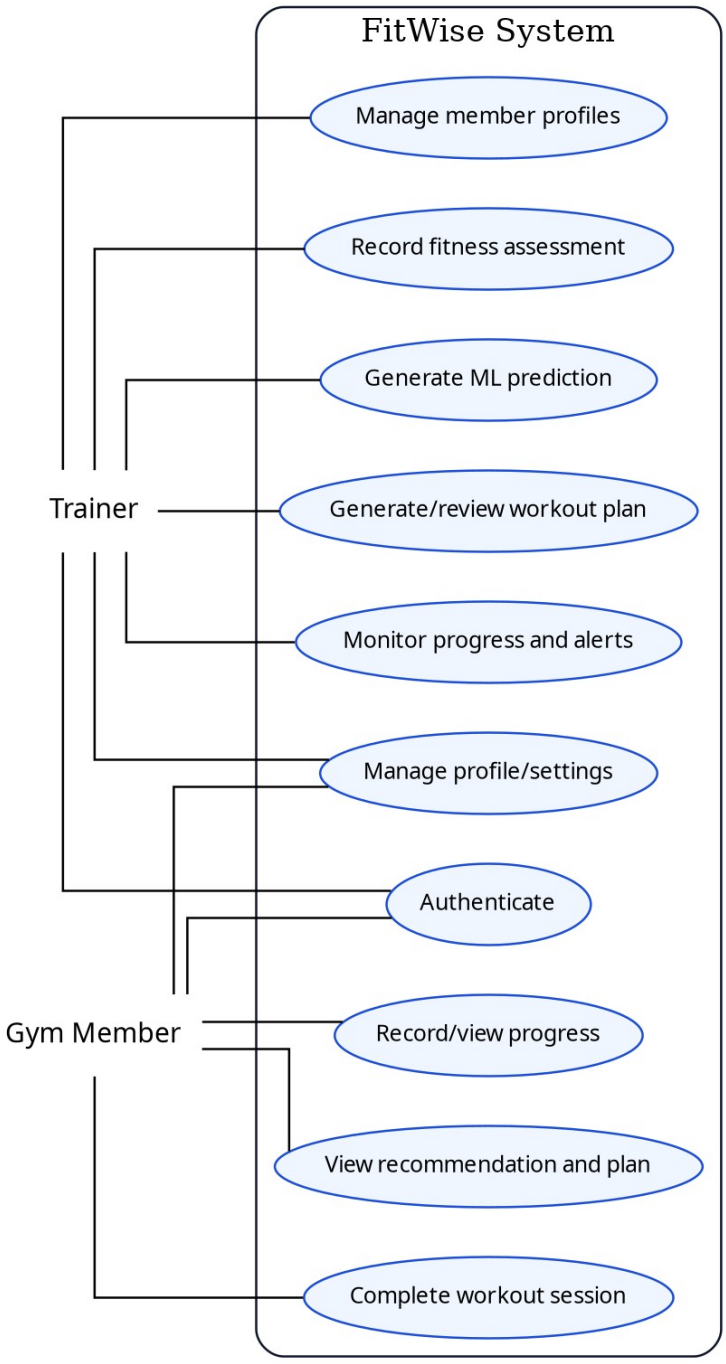


Figure 4.2: FitWise use-case model

Source: Author's design

4.6 Detailed Core Use Cases

UC-01 – Create Member and Assessment. The primary actor is the trainer. Preconditions are an authenticated active trainer and no existing account with the

same email. The trainer enters account and assessment information. The server validates types and ranges, creates or invites the Supabase user, stores the profile and creates the trainer–member assignment. If validation fails or a duplicate exists, no partial profile is committed. The postcondition is a member with a complete draft assessment ready for prediction.

UC-02 – Generate Prediction and Draft Plan. The trainer selects a complete assessment and requests a prediction. Next.js verifies assignment and sends the allowed feature schema to FastAPI. The returned category and confidence are stored. Next.js fetches approved exercises, calls Gemini with a structured schema and validates the result. Alternative flows include unavailable ML service, Gemini timeout, schema failure or semantic validation errors. The postcondition is a stored draft plan with validation status, not an automatically assigned plan.

UC-03 – Review and Assign Plan. The trainer opens the draft, examines the prediction explanation and plan content, edits permitted values or rejects/regenerates the plan, and records any override reason. Approval changes the plan status and makes it visible to the member. A plan that has not passed validation cannot be approved.

UC-04 – Complete Workout. The member opens the assigned plan, starts a scheduled workout, records actual sets, repetitions or duration and completes the session. The system verifies ownership and plan status, saves completion records and recalculates adherence. If the network request fails, the client must display an error and avoid falsely marking the workout as synchronised.

UC-05 – Record and Review Progress. An authorised trainer or member records a permitted measurement. The system stores the dated value, updates actual trend charts and recalculates the transparent projection. Both roles see only data permitted by policy. The system displays the projection assumptions and does not present the result as guaranteed.

4.7 User Stories and Acceptance Criteria

Table 4.4: Prioritised user stories

Priority	User Story	Acceptance Summary
Must	As a trainer, I want to create a complete member assessment so that recommendations use consistent information.	Invalid ranges and duplicates are rejected; valid assessment is persisted.
Must	As a trainer, I want to see the category, confidence and contributing information so that I can judge the model output.	Prediction record displays category, scores, model version and timestamp.
Must	As a trainer, I want Gemini to use only approved exercises so that generated plans remain trackable.	Every returned exercise ID exists, is active and was supplied as a candidate.
Must	As a trainer, I want to approve or override a plan so that human supervision is preserved.	Member sees only approved plan; override reason is audited.
Must	As a member, I want to see today's workout and instructions so that I know what to perform.	Current plan day and exercise details load for the authenticated owner.
Must	As a member, I want to record what I completed so that progress reflects actual activity.	Session details persist and adherence updates.
Should	As a trainer, I want low-adherence alerts so that I can follow up with members.	Defined threshold creates a role-appropriate alert.
Should	As a member, I want understandable progress charts so that I can compare actual performance with my target.	Chart labels and assumptions are clear during UAT.
Could	As a member, I want badges and streaks so that regular completion feels rewarding.	Feature added only after core integration and testing are stable.

4.8 Requirements Traceability

Traceability connects each requirement to the project objectives, design components and future test evidence. Status values in this chapter describe planned coverage; they must be replaced by implemented/tested status in Chapter 7.

Table 4.5: Requirements traceability matrix

Requirement	Objective	Design Component	Planned Test	Draft Status
FR-02 Authentication	OBJ-05/06/07	Supabase Auth, session middleware and RLS	TC-AUTH-01 to 05	Designed
FR-03/04 Member and assessment	OBJ-05/07	Profile service, relational schema and trainer assignment	TC-MEM-01 to 08	Designed

Requirement	Objective	Design Component	Planned Test	Draft Status
FR-05 Prediction	OBJ-02/08	FastAPI model service and prediction table	TC-ML-01 to 10	Designed
FR-07/08 Plan draft and validation	OBJ-03/08	Gemini service, Zod schema and deterministic validator	TC-AI-01 to 15	Designed
FR-09 Trainer approval	OBJ-03/05	Plan workflow and audit log	TC-PLAN-01 to 08	Designed
FR-10/11 Mobile plan and completion	OBJ-06/07	Flutter repositories/view models and session endpoints	TC-MOB-01 to 12	Designed
FR-12/13 Progress	OBJ-04/05/06	Progress service and dashboards	TC-PROG-01 to 08	Designed
NFR-01 Security	OBJ-08	JWT verification, RLS and ownership checks	TC-SEC-01 to 12	Designed
NFR-09 AI robustness	OBJ-03/08	Candidate restriction, validation and approval	TC-AI-BND/ CONFLICT	Designed

4.9 Chapter Summary

This chapter converted the project problem and literature findings into testable requirements. It established trainer and member access boundaries, defined the controlled prediction and plan workflow, and added measurable expectations for security, privacy, performance, usability, maintainability and AI robustness. Chapter 5 specifies the architecture that will realise these requirements.

CHAPTER 5: SYSTEM DESIGN AND ARCHITECTURE

5.1 Architectural Overview

FitWise uses a layered client–server architecture with Next.js as the main application and orchestration layer. The public website and trainer web application are implemented in one Next.js App Router project. Route Handlers expose REST endpoints required by Flutter, while Server Actions or server-side services support web mutations and shared business logic. Next.js documentation describes Route Handlers as custom request handlers within the app directory and supports standard HTTP methods, making them suitable for the mobile API surface (Next.js, 2026a).

Supabase provides authentication, PostgreSQL storage, file storage and Row-Level Security. Supabase Auth issues JWTs and integrates with database policies; RLS acts as a database-level authorisation layer and should be enabled on every exposed table (Supabase, 2026a; Supabase, 2026b). The Python model remains a separate runtime because a joblib scikit-learn pipeline cannot be imported directly by TypeScript. A small FastAPI service in the same repository loads the trusted model and exposes prediction and health endpoints. Gemini is called only by the trusted Next.js backend.

This separation gives each component one principal responsibility. Clients render user experiences; Next.js validates and orchestrates; Supabase is the persistent source of truth; FastAPI performs model inference; Gemini composes a draft; and the validator plus trainer determines whether a plan is assigned. A failure in Gemini must not prevent members from accessing already approved plans, and a FastAPI outage must not corrupt stored assessments.

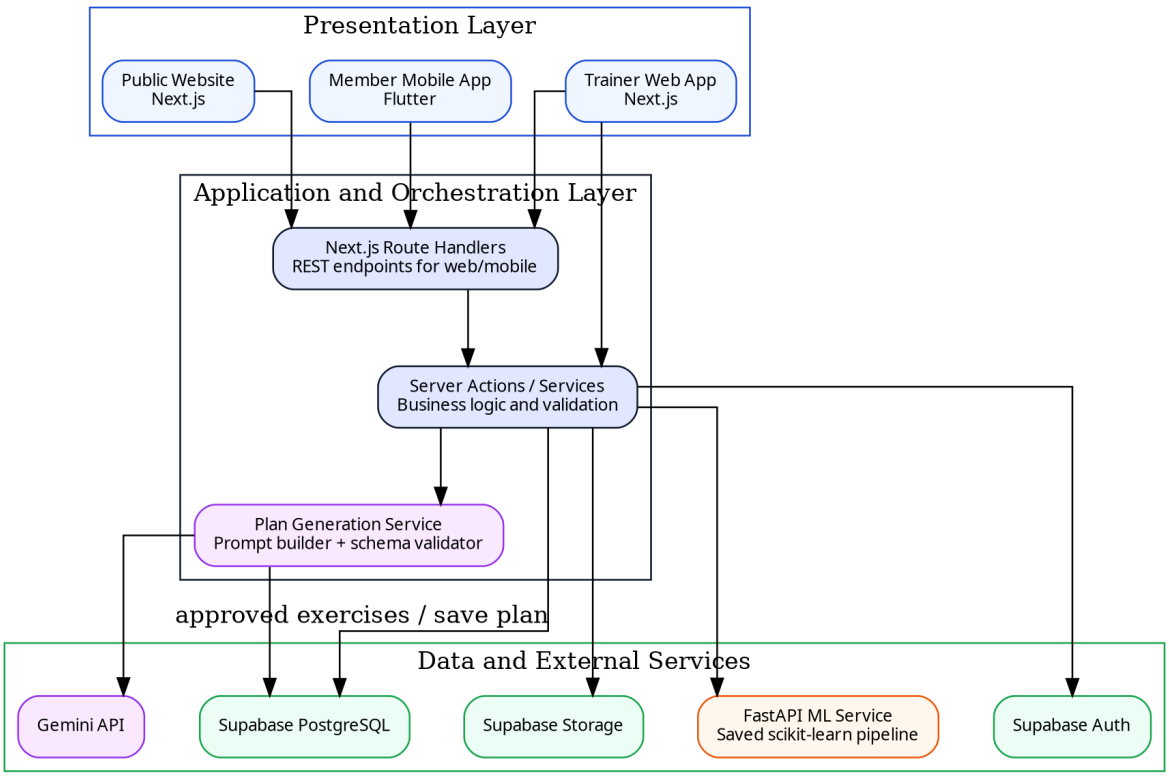


Figure 5.1: FitWise system architecture

Source: Author's design

5.2 Technology Stack and Design Rationale

Table 5.1: Technology stack and design rationale

Layer	Selected Technology	Purpose	Rationale / Alternative
Public and trainer web	Next.js App Router + TypeScript	Website, trainer UI, server rendering and web mutations	Combines frontend and backend concerns in one typed project; selected over separate static frontend plus general API.
Mobile client	Flutter + Dart	Gym-member mobile experience	Cross-platform UI with one codebase; recommended layered architecture improves testability (Flutter, 2026).
Application API	Next.js Route Handlers / server services	REST for Flutter, validation, business logic and orchestration	Avoids duplicating business logic in clients; supports standard HTTP methods (Next.js, 2026a).
Relational data	Supabase PostgreSQL	Profiles, assessments, exercises, predictions, plans, sessions and progress	Relational constraints and joins suit the domain better than the previous Firestore proposal.
Identity and authorisation	Supabase Auth + RLS	JWT identity and row-level access	Auth integrates with PostgreSQL policies; adds defence in depth (Supabase, 2026a; 2026b).
Media storage	Supabase Storage	Profile and exercise media	Storage policies can use RLS;

Layer	Selected Technology	Purpose	Rationale / Alternative
			avoids storing binaries in database rows (Supabase, 2026c).
ML development	Python, pandas and scikit-learn	EDA, preprocessing, model training and export	Mature classification and pipeline ecosystem; consistent with selected dataset and student skills.
ML inference	FastAPI	Load saved pipeline and serve prediction	Lightweight typed Python API; lifespan can load shared model once (FastAPI, 2026a).
Plan drafting	Gemini API structured output	Create weekly draft and explanation from approved exercises	Flexible composition with JSON schema; values remain subject to semantic validation (Google AI for Developers, 2026).
Deployment	Vercel/equivalent + Python container + Supabase Cloud	Host web/API, ML service and data	Separates runtime requirements while preserving a single Git repository and repeatable CI/CD.

5.3 Component Design

Within Next.js, the system should be divided into domain services rather than embedding database and AI calls in page components. The Authentication Service resolves the session and role. The Member Service manages profiles, assignments and assessments. The Prediction Service maps a validated assessment to the FastAPI schema and stores the result. The Exercise Catalogue Service retrieves candidates. The Plan Generation Service constructs prompts, calls Gemini, parses output and invokes the Plan Validator. The Progress Service aggregates sessions and measurements, and the Audit Service records sensitive actions.

The Flutter application will follow the separation recommended by the Flutter architecture guide: views and view models in the UI layer, repositories as the source of application data and services for HTTP/authentication operations (Flutter, 2026). The Flutter client should not contain Gemini or Supabase service-role credentials. It obtains a user session and calls the Next.js API using the bearer token. A repository converts JSON DTOs into typed domain models and exposes loading, success and error states to the view model.

Design principle
Next.js is the orchestration boundary. Neither the browser nor Flutter is allowed to decide that a generated plan is valid or to bypass the trainer-approval workflow.

5.4 Key Interaction and Sequence Design

The prediction and plan sequence begins with an authenticated trainer request. Next.js verifies the trainer–member relationship before storing or reading member data. It sends only the model schema to FastAPI and stores the result with the model version. Candidate exercises are selected from Supabase before Gemini is called. The validator then checks semantic constraints. A plan can remain in generated, validation_failed, draft, approved, assigned, archived or rejected status, preventing an incomplete response from appearing as active.

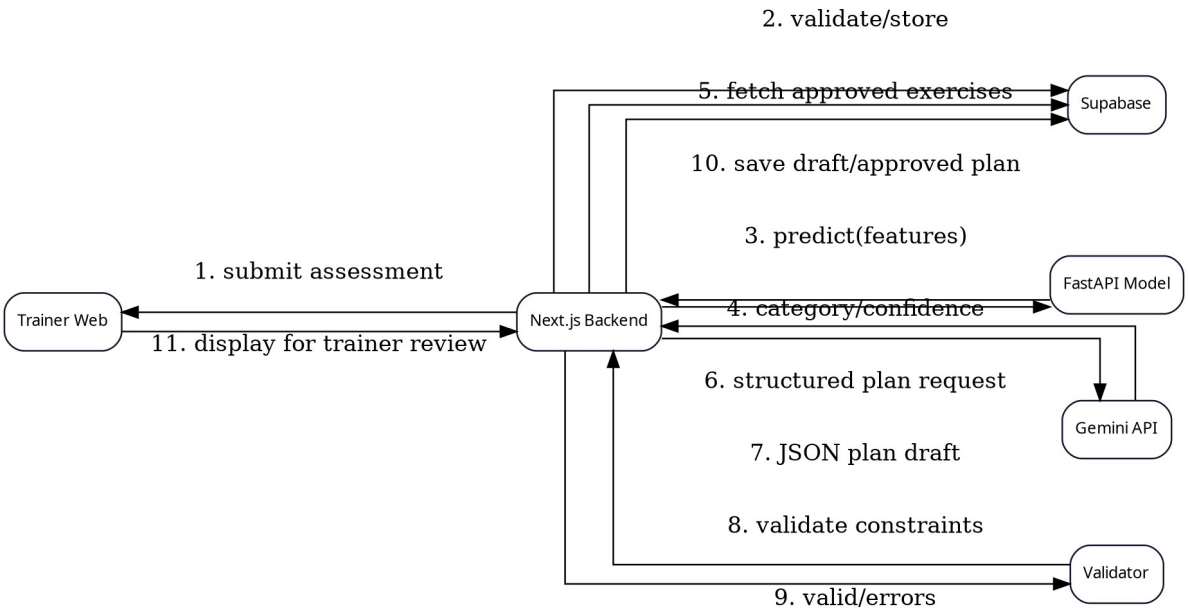


Figure 5.2: Sequence for prediction and plan generation

Source: Author's design

For workout completion, Flutter requests the active plan through Next.js. When the member starts a session, the API creates a workout_sessions record. Exercise completion writes are linked to the session and approved plan-day item. On final completion, the server calculates summary values and adherence in one controlled operation. Repeated submissions should be idempotent through a unique

session/action identifier or a state check, preventing duplicate completion records when a mobile request is retried.

5.5 Data Model and Database Design

A relational model was selected because FitWise contains clear one-to-many and many-to-many relationships. Supabase Auth owns the authentication identity, while public.profiles stores application attributes and references auth.users by the stable UUID. A trainer_member_assignments table defines which trainer may manage which member. Fitness profiles store relatively stable attributes; assessments and progress records preserve dated changes. Predictions reference the assessment and model version used. Workout plans contain plan days and plan exercises that reference catalogue exercises.

The schema separates a generated plan from actual workout sessions. This allows a member to perform the same assigned plan on multiple dates while preserving the originally approved prescription. Audit logs record actor, action, entity and timestamp for sensitive transitions. Foreign keys, not-null constraints, checks and unique indexes should enforce integrity before RLS is considered.

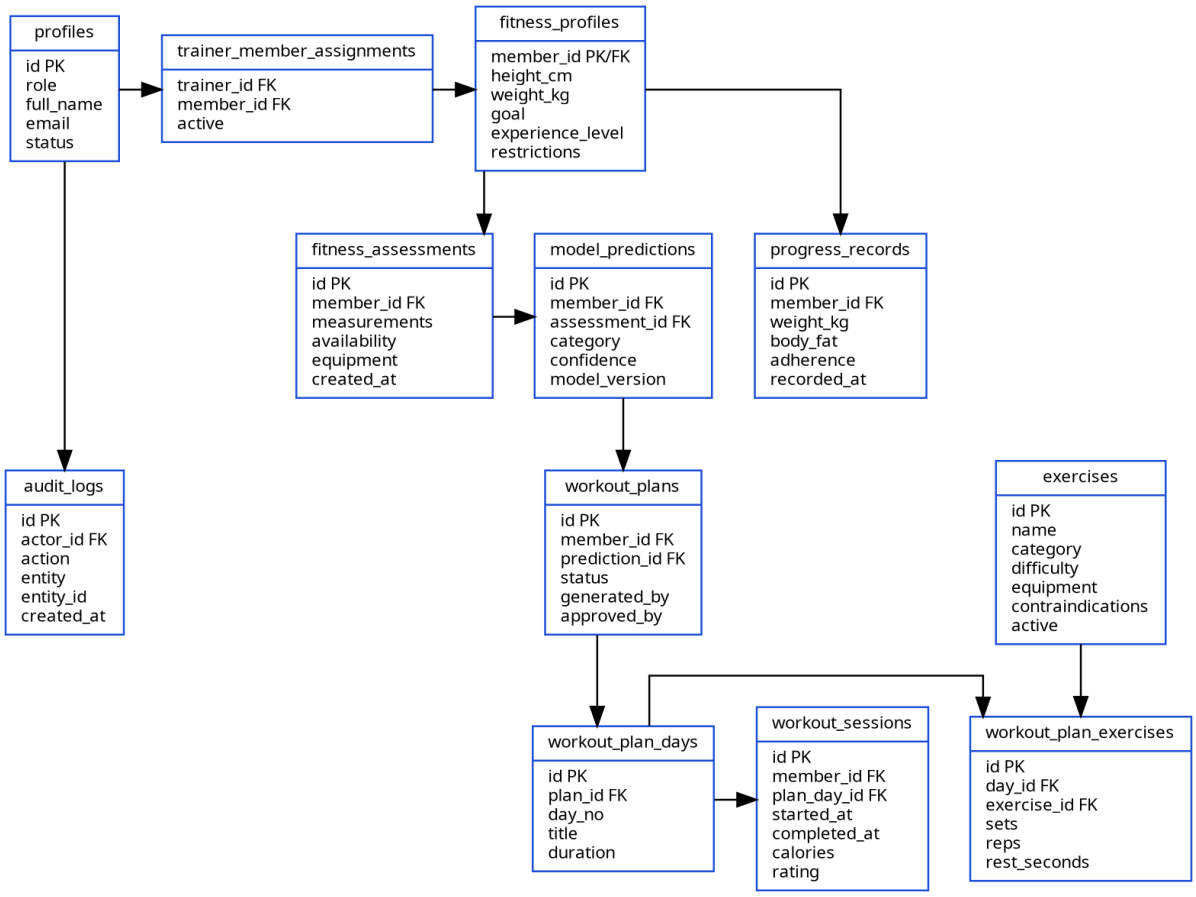


Figure 5.3: Logical relational data model

Source: Author's design

Table 5.2: Core database entities

Entity	Purpose	Selected Fields	Security / Relationship
profiles	Application identity and role	id, full_name, role, status	References auth.users; own profile or authorised trainer view.
trainer_member_assignments	Trainer ownership boundary	trainer_id, member_id, active	Used by RLS and server role checks.
fitness_profiles	Current member context	goal, level, measurements, restrictions	Member read-own; assigned trainer manage.
fitness_assessments	Dated recommendation input	availability, equipment, measurements, created_at	Immutable or versioned after prediction.
model_predictions	ML evidence	category, scores, model_version, assessment_id	Linked to member and requesting trainer.
exercises	Approved exercise catalogue	category, difficulty, equipment, contraindications, active	Read candidates server-side; admin-maintained.

Entity	Purpose	Selected Fields	Security / Relationship
workout_plans / days / exercises	Generated and approved prescription	status, generated_by, approved_by, sets, reps, rest	Member reads assigned approved plan only.
workout_sessions	Actual performed workout	start/end, completion, calories, rating	Member writes own active session; trainer reads assigned.
progress_records	Dated actual measurements	weight, body_fat, adherence, recorded_at	Member/assigned trainer according to policy.
audit_logs	Traceable sensitive actions	actor, action, entity, entity_id, created_at	Insert server-side; restricted read.

5.6 API Design

The public website may use Server Components and Server Actions, while Flutter requires stable HTTP endpoints. Route Handlers will return JSON with consistent success and error structures. All protected endpoints will resolve the Supabase session and then perform explicit role and ownership checks. The service-role key is used only where a trusted server operation genuinely requires it; normal user operations should preserve the signed-in user context so RLS remains effective.

Table 5.3: Proposed API endpoints

Method	Endpoint	Role	Purpose	Key Validation
POST	/api/members	Trainer	Create/invite member and profile	Trainer role, unique email, required fields and transaction/cleanup.
POST	/api/members/{id}/ assessments	Trainer	Create assessment	Assignment, numeric ranges, allowed enums and complete context.
POST	/api/predictions	Trainer	Run ML inference	Assignment, assessment completeness, exact model schema and service timeout.
POST	/api/plans/generate	Trainer	Generate and validate draft	Prediction exists, candidate exercises, rate limit, schema and semantic validator.
PATCH	/api/plans/{id}	Trainer	Edit/approve/reject draft	Assignment, allowed status transition and override reason.
GET	/api/me/plan	Member	Retrieve own approved active plan	Authenticated owner; approved/assigned status only.
POST	/api/workouts/{dayId}/ start	Member	Create workout session	Ownership, active plan and no conflicting active

Method	Endpoint	Role	Purpose	Key Validation
				session.
POST	/api/workout-sessions/{id}/complete	Member	Store completion and summary	Session owner, value bounds and idempotency.
GET	/api/progress/{memberId}	Member/Trainer	Retrieve authorised progress	Owner or assigned trainer; filtered fields.
POST	/api/progress/{memberId}	Member/Trainer	Record measurement	Owner/assignment, allowed fields and range validation.
GET	/api/health/ml	Maintainer	Check inference service	Server/internal access; no sensitive details.

5.7 Machine-Learning Model Design

The ML design has separate training and inference paths. Training begins with an immutable dataset snapshot and data-quality report. The leakage audit produces the final feature list. A ColumnTransformer applies categorical encoding and numeric transformations within each cross-validation fold. Candidate estimators are evaluated, and the selected complete pipeline is exported with its model card and dependency metadata. The FastAPI input schema mirrors the feature list exactly.

At service start-up, the FastAPI lifespan handler loads the trusted model once. The /predict endpoint validates the request, creates the expected tabular structure and returns the predicted class, confidence/probability map and model version. The model file is not exposed for download, and user-supplied model paths are never accepted. Joblib files are trusted build artefacts because pickle-based loading can execute code (Scikit-learn Developers, 2026).



Figure 5.4: Machine-learning training and inference pipeline

Source: Author's design

5.8 Gemini-Assisted Plan Generation and Deterministic Validation

The plan generator receives the saved prediction and member context but does not expose the complete database. The server queries a bounded candidate set and sends exercise IDs, names, category, difficulty, equipment and short constraints. The

prompt asks Gemini to produce a weekly plan matching a strict schema. Temperature should be low, token/output size bounded and the request timed out. The response is parsed through a Zod schema before semantic validation.

The semantic validator is deterministic and versioned. It checks that every exercise was offered as a candidate, that the member has the required equipment, that beginner members do not receive advanced items, and that reported restriction mappings are respected. It calculates estimated duration from warm-up, exercise prescriptions and rest rather than trusting the model's stated total. A valid plan is still a draft until the trainer approves it.

Table 5.4: Deterministic plan-validation rules

Rule ID	Condition	Action	Priority
VAL-01	Exercise ID was not supplied or is inactive	Reject item and plan	Critical
VAL-02	Required equipment is unavailable	Reject or substitute from approved candidate set	Critical
VAL-03	Exercise matches a configured restriction/exclusion	Reject and require trainer review/substitution	Critical
VAL-04	Difficulty exceeds member level	Reject item unless trainer explicitly overrides	High
VAL-05	Session count exceeds available days	Reject plan	High
VAL-06	Calculated duration exceeds allowed duration plus tolerance	Return validation error/repair request	High
VAL-07	Sets, repetitions, timed duration or rest outside configured bounds	Clamp only if rule is explicit; otherwise reject	High
VAL-08	Duplicate exercise appears inappropriately within a session	Reject duplicate or require explanation	Medium
VAL-09	Weekly plan lacks balance for selected category/template	Flag for trainer review	Medium
VAL-10	Trainer override provided	Apply permitted change and store reason/actor	Highest human decision

5.9 Security and Privacy Design

Supabase Auth establishes user identity through JWT sessions. Application roles should be stored in trusted app metadata or a protected profile/role table, not editable user metadata. Next.js verifies the session for every protected request and performs function-level checks before accessing business services. RLS is enabled on all exposed public tables. A member policy uses auth.uid() to match the member profile or owner foreign key; a trainer policy additionally checks an active trainer_member_assignments relationship. Supabase recommends explicit RLS on exposed tables and notes that service keys bypass policies and must never be placed in the browser (Supabase, 2026b).

The architecture applies defence in depth. A bug in a page query should still encounter RLS, while the backend independently prevents broken object-level authorisation. The browser and Flutter receive only publishable configuration and user sessions. Gemini and service-role keys remain in server environments. Supabase Storage policies restrict profile or exercise uploads to approved buckets and paths. Audit logging records plan approval, override and sensitive record changes.

Input validation uses bounded numeric schemas, allowed enums and UUID validation. External API responses are treated as untrusted. Generation endpoints are rate-limited per trainer and member, and request content is minimised. Logs omit passwords, tokens, API keys and unnecessary fitness details. HTTPS is required for deployed web, mobile API and FastAPI communication. Development CORS settings should list explicit origins rather than allow unrestricted production access.

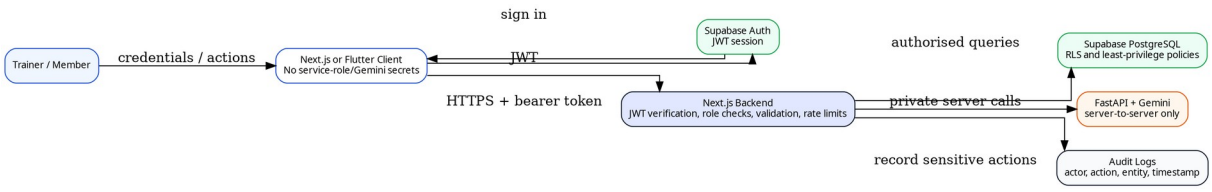


Figure 5.5: Security boundaries and access-control design

Source: Author's design

Table 5.5: Access-control matrix

Resource / Action	Member	Assigned Trainer	Other Trainer	Anonymous
View member profile	Own only	Assigned member	Denied	Denied
Edit fitness assessment	Limited own fields if enabled	Allowed for assigned member	Denied	Denied
Request ML prediction	Denied	Allowed for assigned member	Denied	Denied
Generate/approve plan	Denied	Allowed for assigned member	Denied	Denied
View approved plan	Own only	Assigned member	Denied	Denied
Record workout session	Own only	Read assigned member	Denied	Denied
View progress	Own only	Assigned member	Denied	Denied
Manage exercise catalogue	Denied	Denied unless explicit maintainer role	Denied	Denied

5.10 UI/UX and Navigation Design

The public website introduces the problem, explains the workflow and provides authentication/download entry points. The trainer web application uses a persistent sidebar because trainers frequently move between dashboard, members, prediction, plan and progress views. The member mobile application uses bottom navigation for Home, Plan, Workouts, Progress and Profile. The POC includes additional screens, but implementation priority remains the core end-to-end scenario.

Interface design uses a consistent FitWise blue, teal and green palette, readable cards and clear status labels. Prediction confidence is accompanied by a plain-language explanation and disclaimer. Generated plans distinguish draft, validation-failed, approved and assigned states. Empty, loading and failure states are required so that external-service errors are not mistaken for a valid zero result. Mobile controls will use touch-friendly sizes and scrolling for forms or long workout details.

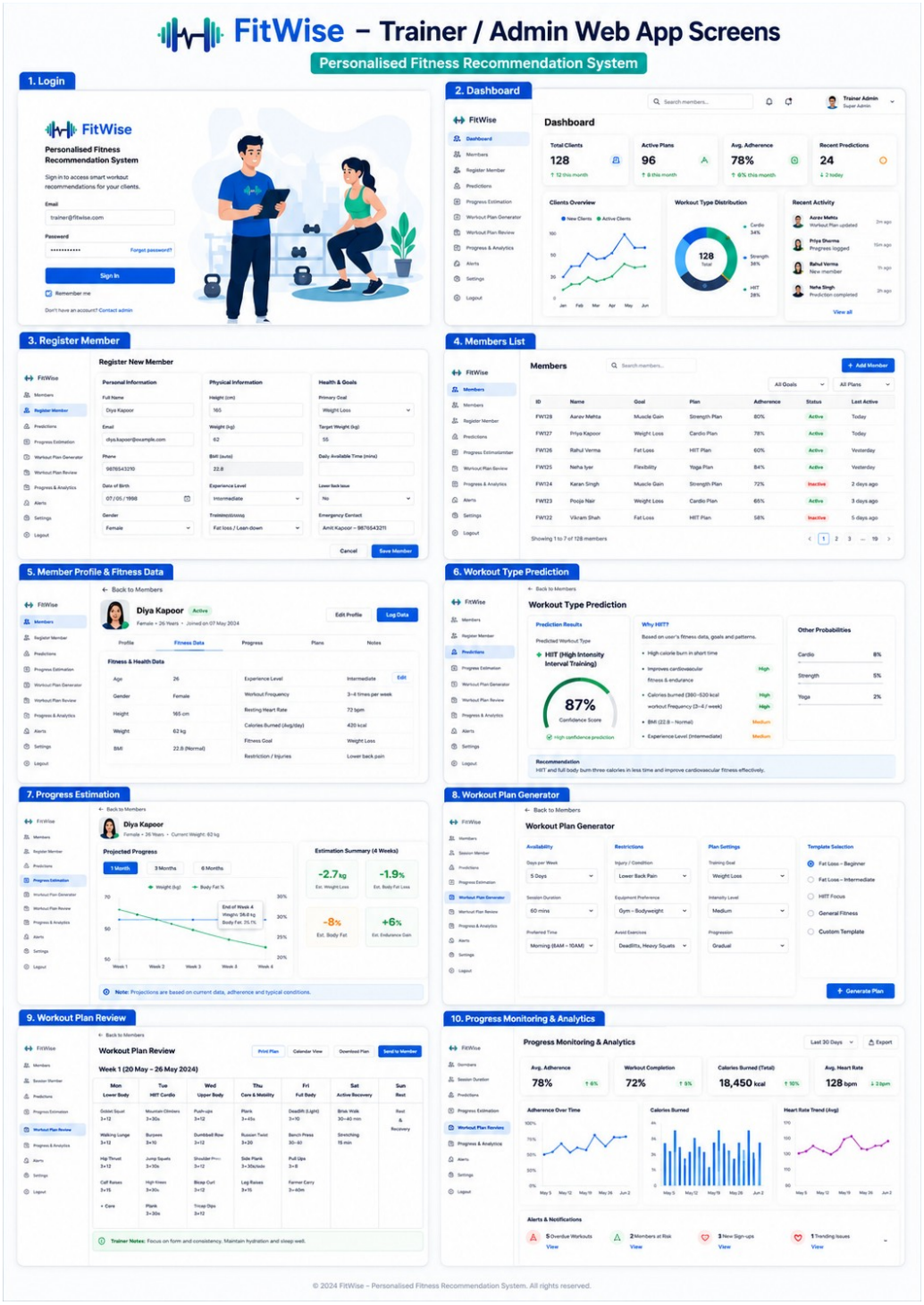


Figure 5.6: Trainer web-application POC screen gallery

Source: Author's FitWise POC

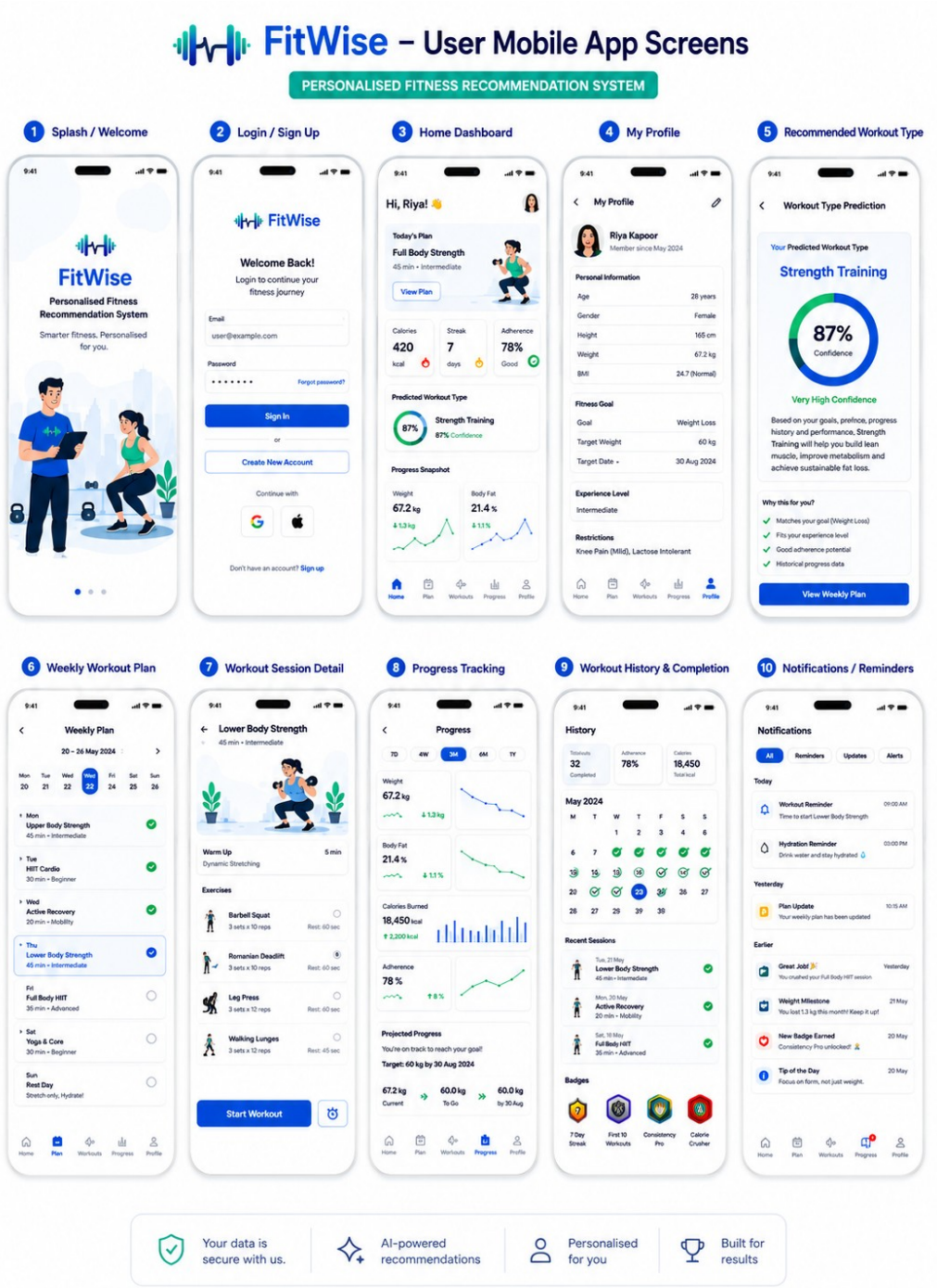


Figure 5.7: Gym-member mobile-application POC screen gallery

Source: Author's FitWise POC

5.11 Deployment Architecture

The project can remain in one Git repository with /web, /mobile, /ml-service and /docs folders while using separate runtime deployments. The Next.js application may be hosted on Vercel or an equivalent Node-compatible platform. The FastAPI service will be deployed as a Python container or managed Python service with a health endpoint and the model artefact packaged from the trusted repository. Supabase hosts PostgreSQL, Auth and Storage. Flutter will produce an Android APK for demonstration and may later support iOS builds.

Environment variables will include Supabase URL and publishable key for clients, server-side Supabase/service credentials where required, ML service URL and internal key, and the Gemini API key. Values will not be committed. CI/CD should run linting and tests before deployment. The ML deployment must pin Python, scikit-learn, numpy and pandas versions because persisted models require compatible environments. FastAPI deployment should account for HTTPS, restarts, process memory and start-up loading (FastAPI, 2026b).

Operational resilience for the POC includes database export or migration backup, retained model artefacts, seed/demo data, a local development mode and a recorded fallback demonstration. Already approved plans remain stored in Supabase and can be displayed even if Gemini is temporarily unavailable. Generation and inference failures are visible as service errors and never converted silently into default plans.

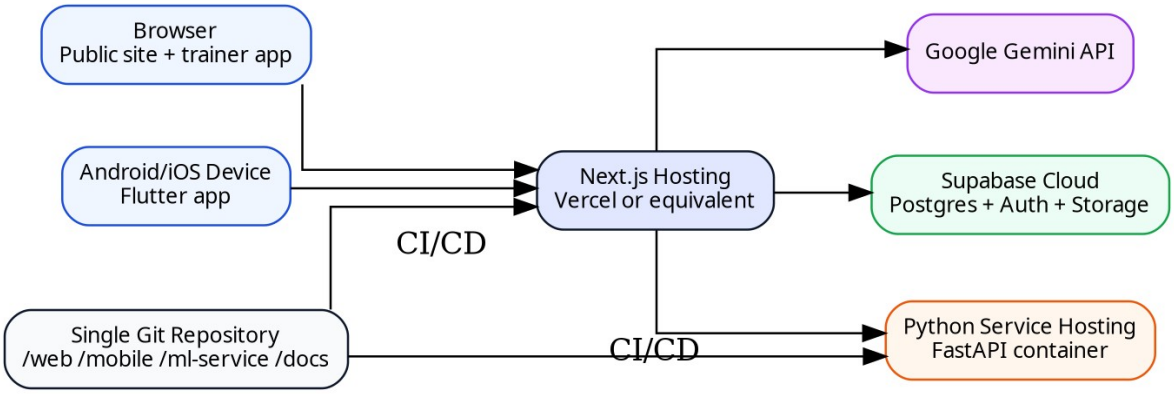


Figure 5.8: FitWise deployment architecture

Source: Author's design

5.12 Chapter Summary

This chapter defined a coherent architecture for the updated FitWise technology stack. Next.js is the central orchestration layer, Supabase is the protected source of truth, FastAPI serves the trusted Python model, Gemini creates only a structured draft, deterministic validation enforces application constraints, and the trainer remains responsible for approval. The relational schema, API contracts, RLS strategy, UI flows and deployment topology provide the basis for implementation. The next chapter of the final dissertation must replace design intentions with actual repository structure, screenshots, database migrations, model outputs and integrated evidence.

REFERENCES

Adomavicius, G. and Tuzhilin, A. (2005) ‘Toward the next generation of recommender systems: a survey of the state-of-the-art and possible extensions’, IEEE Transactions on Knowledge and Data Engineering, 17(6), pp. 734–749. https://doi.org/10.1109/TKDE.2005.99.

Ainsworth, B.E. et al. (2011) ‘2011 Compendium of Physical Activities: A second update of codes and MET values’, Medicine & Science in Sports & Exercise, 43(8), pp. 1575–1581. https://doi.org/10.1249/MSS.0b013e31821ece12.

Alhuwail, D. et al. (2024) ‘Patients’ perspectives on the data confidentiality, privacy, and security of mHealth apps: systematic review’, Journal of Medical Internet Research, 26, e50715. https://doi.org/10.2196/50715.

Aljedaani, B. and Babar, M.A. (2021) ‘Challenges with developing secure mobile health applications: systematic review’, JMIR mHealth and uHealth, 9(6), e15654. https://doi.org/10.2196/15654.

Autio, C. et al. (2024) Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1. Gaithersburg, MD: National Institute of Standards and Technology. https://doi.org/10.6028/NIST.AI.600-1.

Buitinck, L. et al. (2013) ‘API design for machine learning software: experiences from the scikit-learn project’, ECML PKDD Workshop: Languages for Data Mining and Machine Learning, pp. 108–122. Available at: https://arxiv.org/abs/1309.0238 (Accessed: 30 July 2026).

Burke, R. (2002) ‘Hybrid recommender systems: survey and experiments’, User Modeling and User-Adapted Interaction, 12(4), pp. 331–370. https://doi.org/10.1023/A:1021240730564.

Choi, M., Park, J., Lee, M., Beom, J., Jung, S.Y. and Lee, K. (2026) ‘AI-generated exercise prescriptions for at-risk populations: safety and feasibility of a large language model assessed by expert evaluation’, Journal of Clinical Medicine, 15(6), 2457. https://doi.org/10.3390/jcm15062457.

Davergne, T., Meidinger, P., Dechartres, A. and Gossec, L. (2023) ‘The effectiveness of digital apps providing personalized exercise videos: systematic review with meta-analysis’, Journal of Medical Internet Research, 25, e45207. https://doi.org/10.2196/45207.

Emberson, M.A. et al. (2021) ‘Effectiveness of smartphone-based physical activity interventions on individuals’ health outcomes: a systematic review’, BioMed Research International, 2021, 6296896. https://doi.org/10.1155/2021/6296896.

European Commission (n.d.) Information for individuals: special categories of personal data. Available at: https://commission.europa.eu/law/law-topic/data-protection/information-individuals_en (Accessed: 30 July 2026).

European Data Protection Board (n.d.) Basic principles. Available at: https://www.edpb.europa.eu/topics/key-gdpr-concepts/basic-principles_en (Accessed: 30 July 2026).

FastAPI (2026a) Lifespan events. Available at: https://fastapi.tiangolo.com/advanced/events/ (Accessed: 30 July 2026).

FastAPI (2026b) Deployment concepts. Available at: https://fastapi.tiangolo.com/deployment/concepts/ (Accessed: 30 July 2026).

Fitbod (2026) Understanding Fitbod and how it works. Available at: https://help.fitbod.me/hc/en-us/sections/360001078993-Understanding-Fitbod-How-It-Works (Accessed: 30 July 2026).

Flutter (2026) Guide to app architecture. Available at: https://docs.flutter.dev/app-architecture/guide (Accessed: 30 July 2026).

Freeletics (2026) Get started with Freeletics Training. Available at: https://help.freeletics.com/hc/en-us/articles/115004675229-Get-started-with-Freeletics-Training (Accessed: 30 July 2026).

Garber, C.E. et al. (2011) ‘Quantity and quality of exercise for developing and maintaining cardiorespiratory, musculoskeletal, and neuromotor fitness in apparently healthy adults’, Medicine & Science in Sports & Exercise, 43(7), pp. 1334–1359. https://doi.org/10.1249/MSS.0b013e318213fefb.

Google AI for Developers (2026) Structured outputs: Gemini API. Available at: https://ai.google.dev/gemini-api/docs/structured-output (Accessed: 30 July 2026).

He, T., Lu, D., Ma, Y., He, J., Li, D., Li, G. and Sun, J. (2026) ‘The AI recommendation paradox: a systematic review evaluating the promise, peril, and path forward for large language models in exercise recommendation’, Biology of Sport, 43, pp. 949–961. https://doi.org/10.5114/biolsport.2026.158676.

Herlocker, J.L., Konstan, J.A., Terveen, L.G. and Riedl, J.T. (2004) ‘Evaluating collaborative filtering recommender systems’, ACM Transactions on Information Systems, 22(1), pp. 5–53. https://doi.org/10.1145/963770.963772.

Khorasani, V. (2024) Gym Members Exercise Dataset. Kaggle. Available at: https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset (Accessed: 30 July 2026).

Kierner, S., Kucharski, J. and Kierner, Z. (2023) ‘Taxonomy of hybrid architectures involving rule-based reasoning and machine learning in clinical decision systems: a scoping review’, Journal of Biomedical Informatics, 144, 104428. https://doi.org/10.1016/j.jbi.2023.104428.

Laranjo, L. et al. (2021) ‘Do smartphone applications and activity trackers increase physical activity in adults? Systematic review, meta-analysis and metaregression’, British Journal of Sports Medicine, 55(8), pp. 422–432. https://doi.org/10.1136/bjsports-2020-102892.

Liu, X. et al. (2022) ‘Privacy-preserving personalized fitness recommender system (P3FitRec): a multi-level deep learning approach’. Available at: https://arxiv.org/abs/2203.12200 (Accessed: 30 July 2026).

Mahyari, A. and Pirolli, P. (2021) ‘Physical exercise recommendation and success prediction using interconnected recurrent neural networks’. Available at: https://arxiv.org/abs/2010.00482 (Accessed: 30 July 2026).

Michie, S., van Stralen, M.M. and West, R. (2011) ‘The behaviour change wheel: a new method for characterising and designing behaviour change interventions’, Implementation Science, 6, 42. https://doi.org/10.1186/1748-5908-6-42.

Michie, S. et al. (2013) ‘The Behavior Change Technique Taxonomy (v1) of 93 hierarchically clustered techniques’, Annals of Behavioral Medicine, 46(1), pp. 81–95. https://doi.org/10.1007/s12160-013-9486-6.

Next.js (2026a) Route Handlers. Available at: https://nextjs.org/docs/app/getting-started/route-handlers (Accessed: 30 July 2026).

Next.js (2026b) App Router. Available at: https://nextjs.org/docs/app (Accessed: 30 July 2026).

OWASP (2023) OWASP API Security Top 10 – 2023. Available at: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (Accessed: 30 July 2026).

Paganini, S. et al. (2021) ‘Quality of physical activity apps: systematic search in app stores and content analysis’, JMIR mHealth and uHealth, 9(6), e22587. https://doi.org/10.2196/22587.

Pedregosa, F. et al. (2011) ‘Scikit-learn: machine learning in Python’, Journal of Machine Learning Research, 12, pp. 2825–2830.

Ricci, F., Rokach, L. and Shapira, B. (2011) ‘Introduction to recommender systems handbook’, in Ricci, F. et al. (eds.) Recommender Systems Handbook. Boston, MA: Springer, pp. 1–35. https://doi.org/10.1007/978-0-387-85820-3_1.

Scikit-learn Developers (2026) Model persistence. Available at: https://scikit-learn.org/stable/model_persistence.html (Accessed: 30 July 2026).

Shin, D., Hsieh, G. and Kim, Y.-H. (2025) 'PlanFitting: personalized exercise planning with large language model-driven conversational agent', Proceedings of the 7th ACM Conference on Conversational User Interfaces. https://doi.org/10.1145/3719160.3736607.

Strava (2026) Goals and training plans. Available at: https://support.strava.com/en-us/collections/19668882-goals-and-training-plans (Accessed: 30 July 2026).

Supabase (2026a) Auth. Available at: https://supabase.com/docs/guides/auth (Accessed: 30 July 2026).

Supabase (2026b) Row Level Security. Available at: https://supabase.com/docs/guides/database/postgres/row-level-security (Accessed: 30 July 2026).

Supabase (2026c) Storage access control. Available at: https://supabase.com/docs/guides/storage/security/access-control (Accessed: 30 July 2026).

Tong, H.L. et al. (2021) 'Personalized mobile technologies for lifestyle behavior change: a systematic review, meta-analysis, and meta-regression', Preventive Medicine, 148, 106532. https://doi.org/10.1016/j.ypmed.2021.106532.

Wang, L. et al. (2024) 'Applications and concerns of ChatGPT and other conversational large language models in health care: systematic review', Journal of Medical Internet Research, 26, e22769. https://doi.org/10.2196/22769.

World Health Organization (2020) WHO guidelines on physical activity and sedentary behaviour. Geneva: World Health Organization.

World Health Organization (2024) Physical activity. Available at: https://www.who.int/news-room/fact-sheets/detail/physical-activity (Accessed: 30 July 2026).

Yang, Y., Hu, H. and Koenigstorfer, J. (2022) 'Effects of gamified smartphone applications on physical activity: a systematic review and meta-analysis', American Journal of Preventive Medicine, 62(4), pp. 602–613. https://doi.org/10.1016/j.amepre.2021.10.005.

Next completion step

Chapter 6 must be written from the actual repository, database migrations, model experiments, Gemini validation results and integrated screenshots. Chapter 7 must contain executed tests, not expected results. The abstract and conclusion should be revised only after this evidence exists.